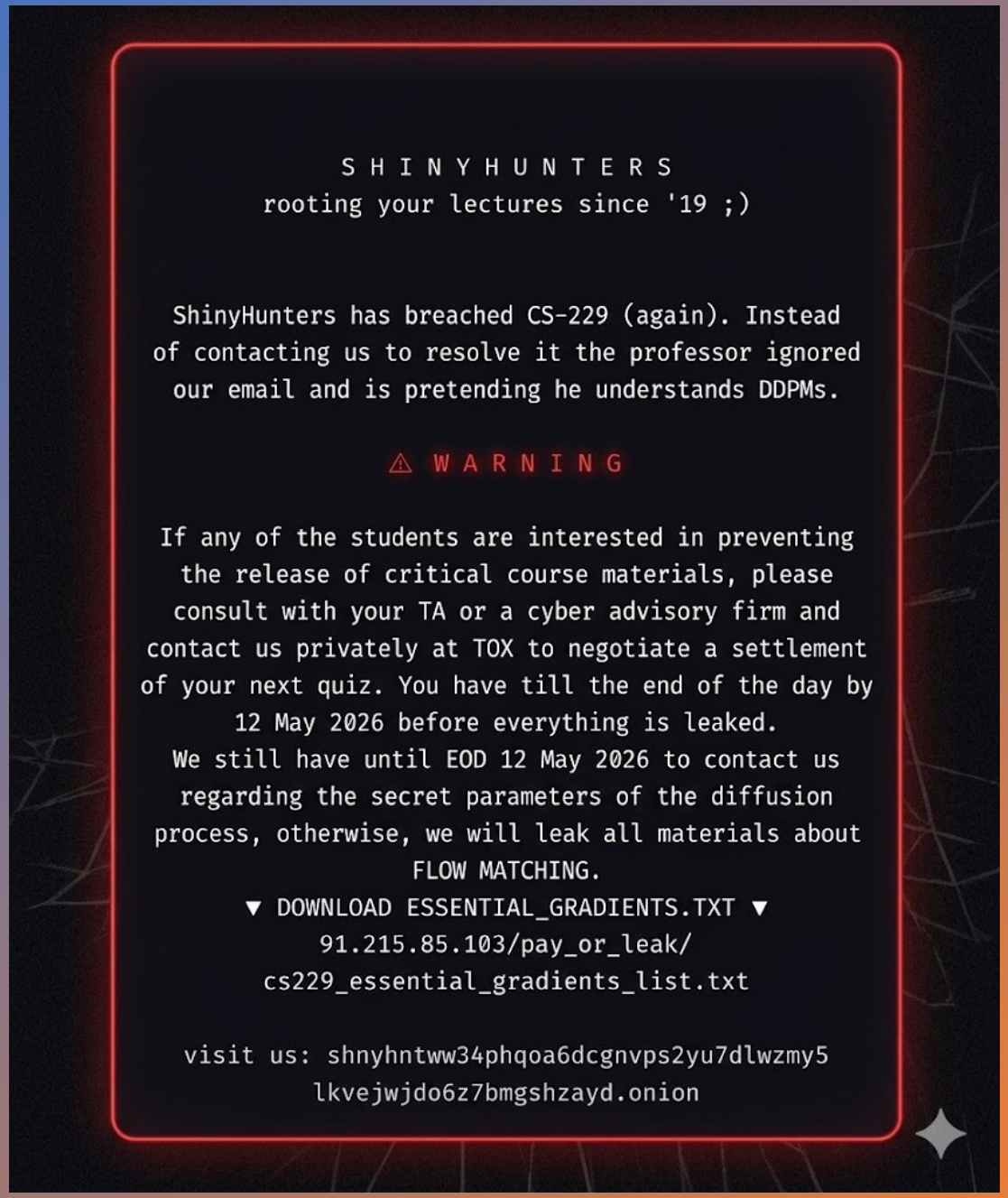


Today: Diffusion! “flow matching” perspective

Later:

- Other diffusion perspectives
- VAEs
- GANs

Links - flow matching

https://arxiv.org/pdf/2511.13720 “Let denoisers denoise” - HW 3 will be based on this paper that simplifies flow matching (I think)

https://arxiv.org/pdf/2412.06264 Canonical tutorial

https://peterroelants.github.io/posts/flow_matching_intro/ - some nice figures

https://diffusion.csail.mit.edu/2026/index.html MIT course

https://mariogemoll.com/flow-matching nice animations

https://arxiv.org/abs/2602.18428 No noise conditioning needed?

Classic Refs on diffusion

- **Jascha** Sohl-Dickstein et al. “Deep Unsupervised Learning using **Nonequilibrium** Thermodynamics.” ICML 2015.
- Yang Song & Stefano Ermon. “Generative modeling by estimating gradients of the data distribution.” NeurIPS 2019.
- Yang Song & Stefano Ermon. “Improved techniques for training **score-based** generative models.” NeuriPS 2020.
- **DDPM:** Jonathan Ho et al. “Denoising diffusion probabilistic models.” arxiv Preprint arxiv:2006.11239 (2020). [code]
- **DDIM:** Jiaming Song et al. “Denoising diffusion implicit models.” arxiv Preprint arxiv:2010.02502 (2020). [code]
- **DDIM+** Alex Nichol & Prafulla Dhariwal. “ Improved denoising diffusion probabilistic models” arxiv Preprint arxiv:2102.09672 (2021). [code]
- Prafula Dhariwal & Alex Nichol. “Diffusion Models Beat GANs on Image Synthesis." arxiv Preprint arxiv:2105.05233 (2021). [code]
- EDM, Karras et al. https://arxiv.org/abs/2206.00364 (Thorough design study)

Plus many more recent refs in the extra slides at the end

MIT course

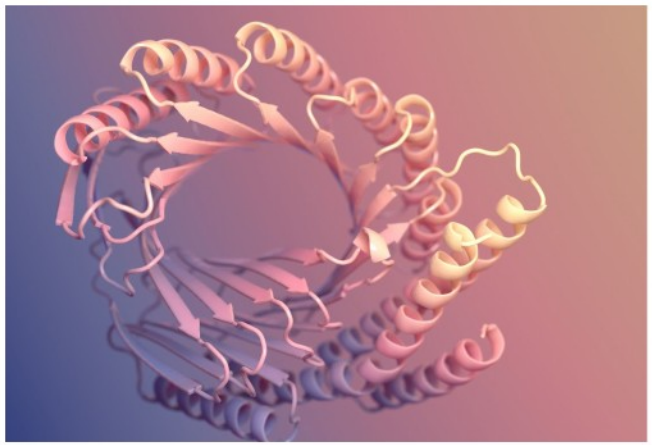
Flow and Diffusion Models: state-of-the-art models for generating images, videos, proteins!

Stable Diffusion

DALEE

OpenAI Sora

Meta MovieGen



AlphaFold3

Boltz

Most SOTA generative AI models for images/videos/proteins/robotics: Diffusion and Flow Models

MIT course

Generative AI - A new generation of AI systems

Artistic Images

Realistic Videos

ChatGPT

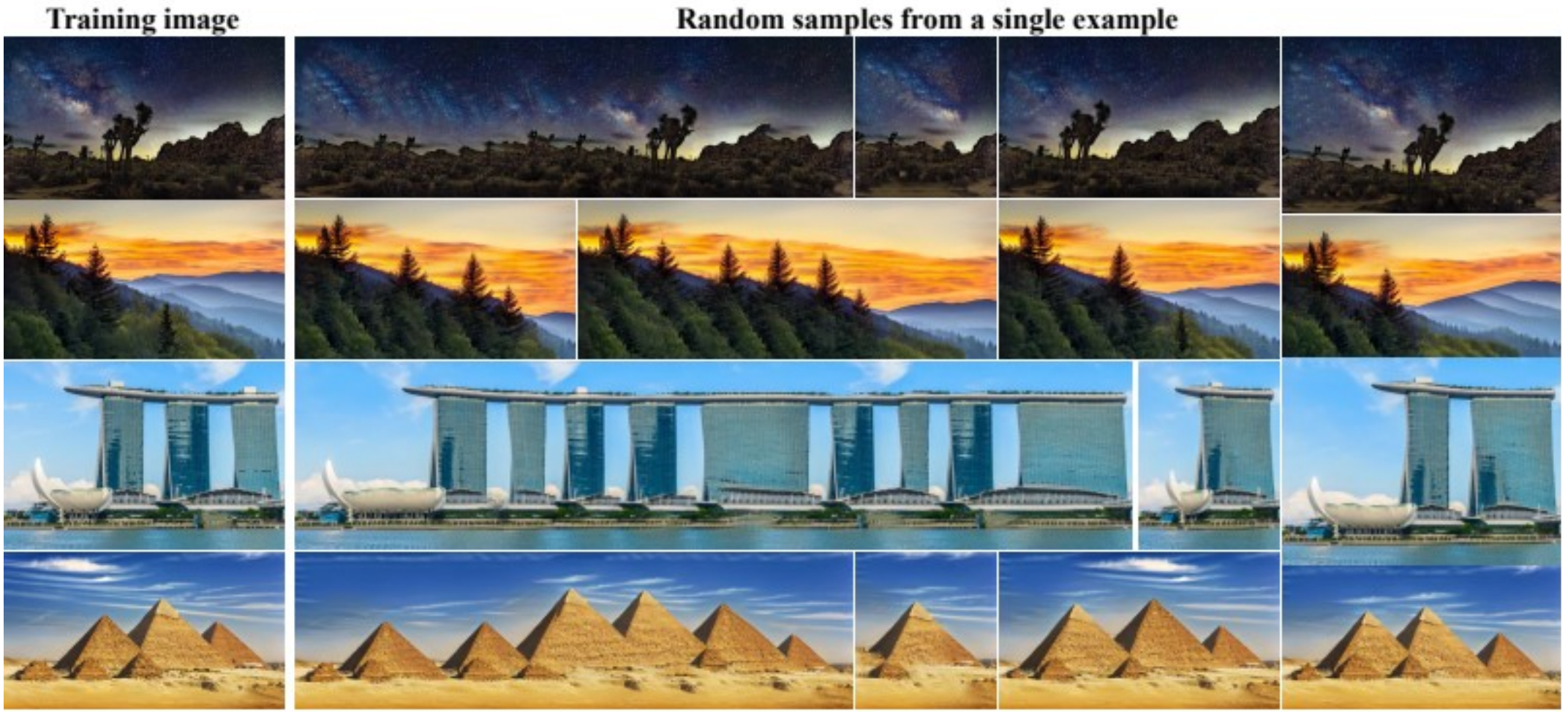
Draft Texts

These systems are “creative”: they generate new objects.

https://icml.cc/virtual/2025/poster/44336 paper about “creativity” of diffusion

This class teaches you algorithms to generate objects.

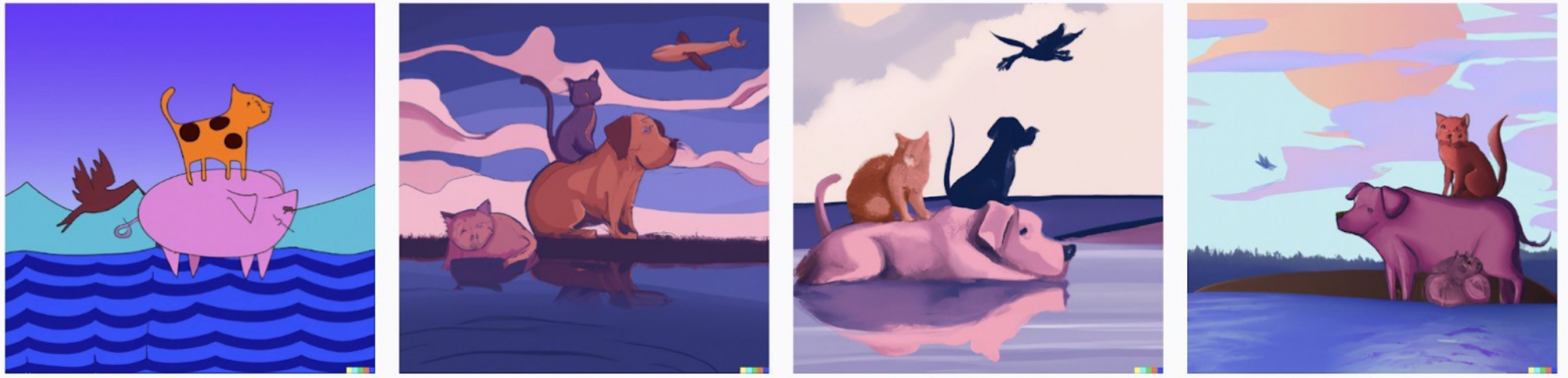
A paper training on a single image with a U-Net



Arxiv:2211.16582

2022 A Bet On compo si-tionali ty

Here, for example, is what DALLE-2 generates if you ask it to draw ***an illustration of a red cat sitting on top of a blue dog next to a purple lake, with a black pig flying in the sky.***



None of the illustrations has a blue dog.
None has a pig flying in the sky! We get birds and planes instead.
Only one of the cats is red.
The cat is sitting on the pig in two of the pictures, not the dog.”

To solve “compositionality” :
New methods or bigger model/ more data?

https://www.surgehq.ai//blog/dall-e-vs-imagen-and-evaluating-astral-codex-tens-3000-ai-bet

Gary Marcus – Slate Star Codex

(2023 dall-e-3)

an illustration of a

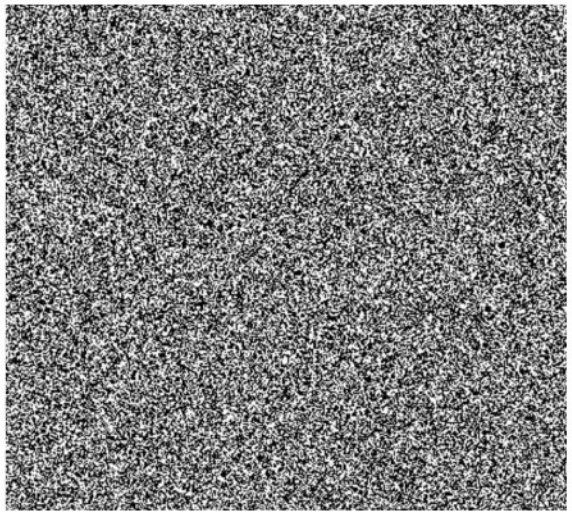
- ✔ **red cat**
- ✔ **sitting on top of**
- ✔ **a blue dog**
- ? **next to**
- ✔ **a purple lake,**
- ✔ **with a black pig**
- ? **flying**
- ✔ **in the sky**

2026: Nano banana (Gemini)

an illustration of a **red cat** (kind of orange?) **sitting on top of a blue dog next to a purple lake, with a black pig flying in the sky**

What does it mean to successfully generate something?

Prompt: “A picture of a dog”

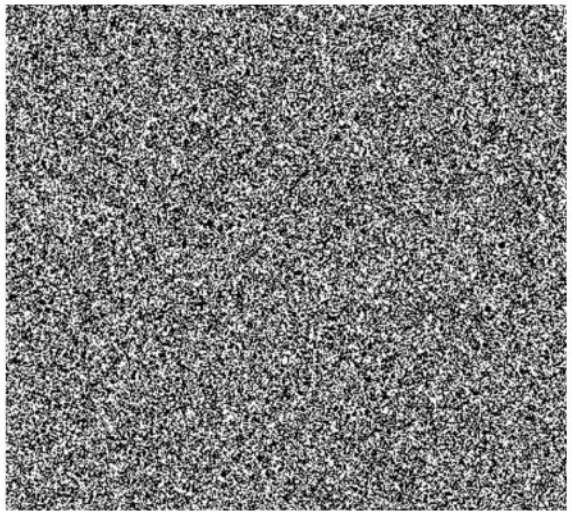



Useless < Bad < Wrong animal < Great!

These are subjective statements - Can we formalize this?

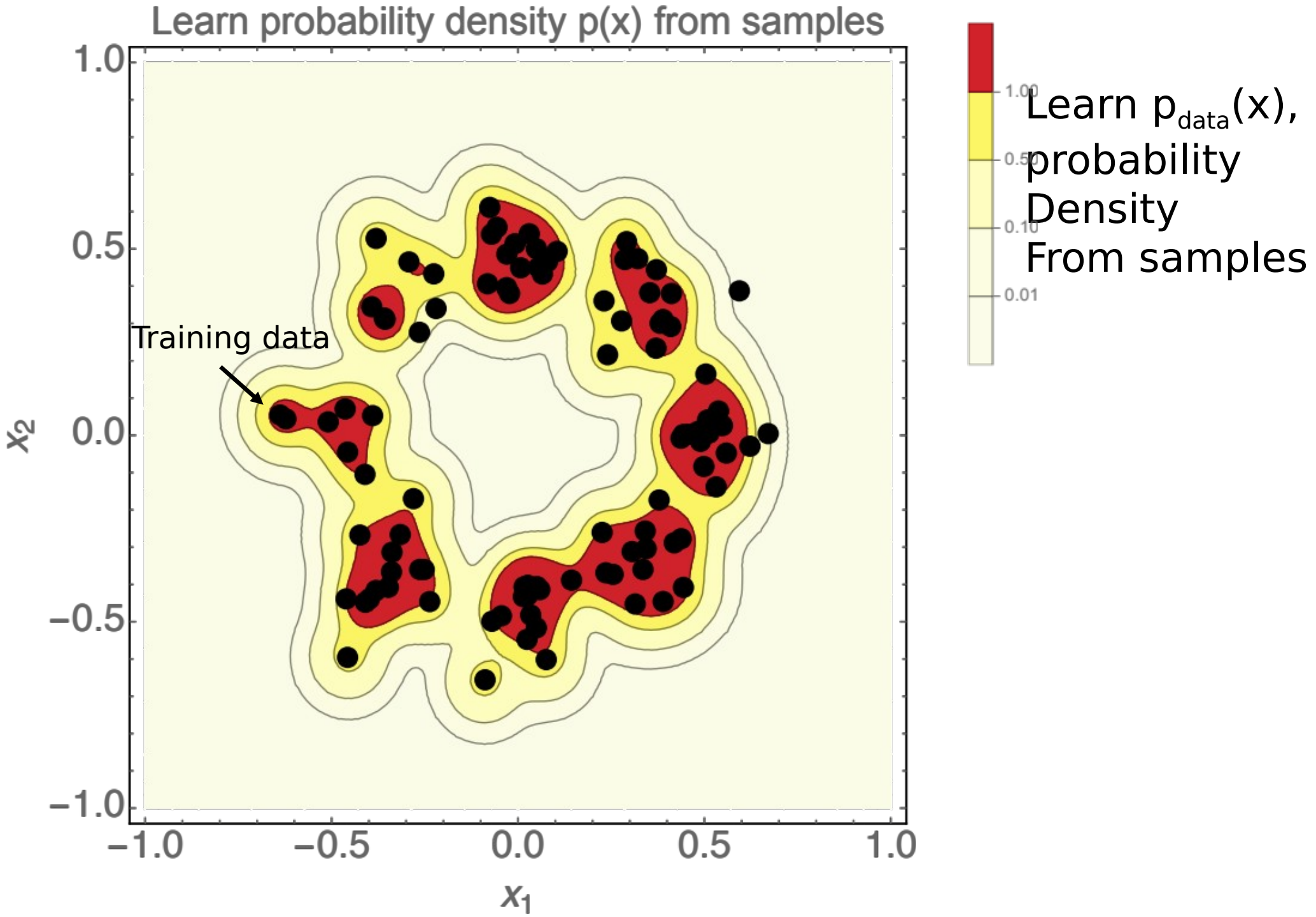
Data Distribution: How “likely” are we to find this picture in the internet?

Prompt: “A picture of a dog”




Impossible < Rare < Unlikely < Very likely

How good an image is ~= How likely it is under the data distribution



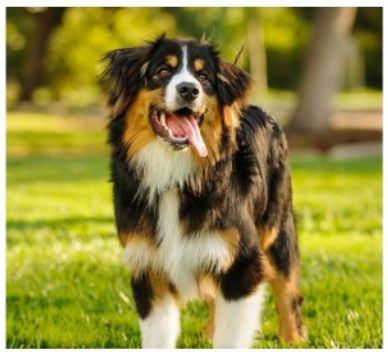
Learn $p_{data}(x)$, probability Density From samples

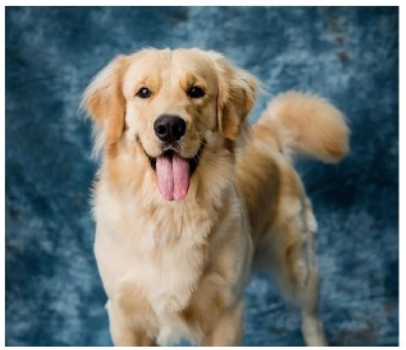
Conditional Generation allows us to condition on prompts

Data distribution p_{data}

Fixed prompt

"Dog"

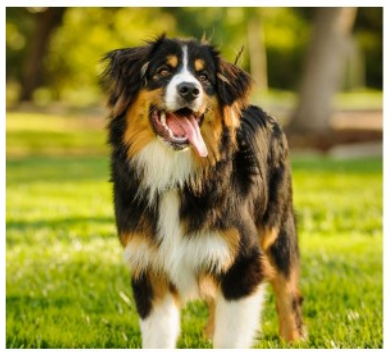




Conditional data distribution $p_{\text{data}}(\cdot|y)$

Condition variable: **y**

y="Dog" y="Cat" y="Landscape"



Conditional generation means sampling the conditional data distribution:

$$z \sim p_{\text{data}}(\cdot|y)$$

We will first focus on unconditional generation and then learn how to ~~translate an unconditional model to a conditional one~~.

How

Most generative models are bridges between easy and hard distribution

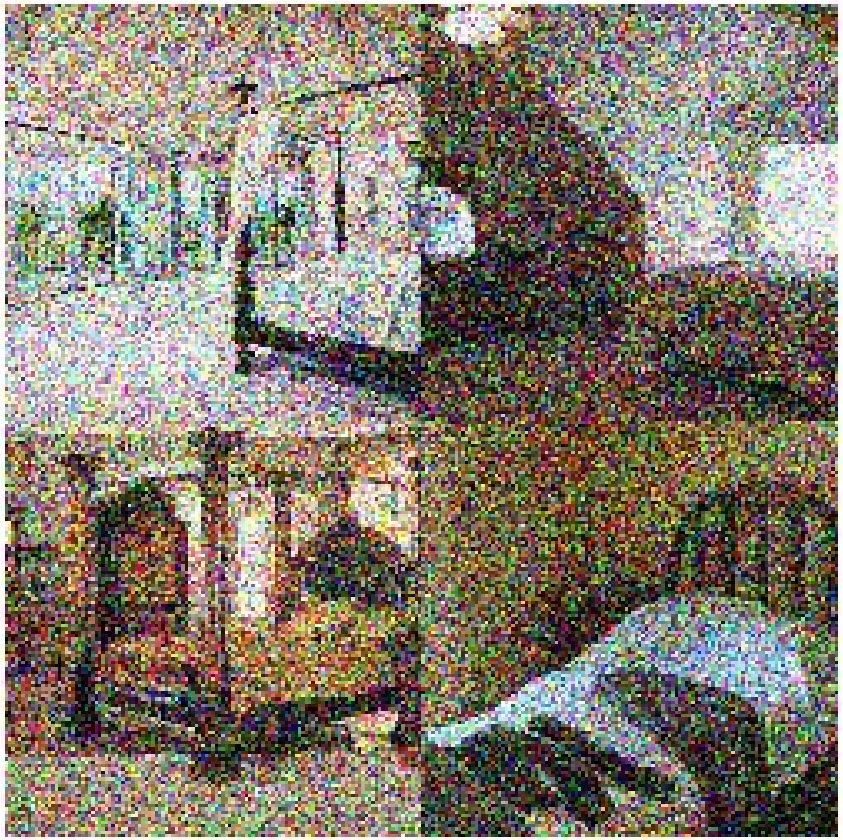
- Normalizing flows
- Diffusion models
- VAEs (kind of)
- GANs

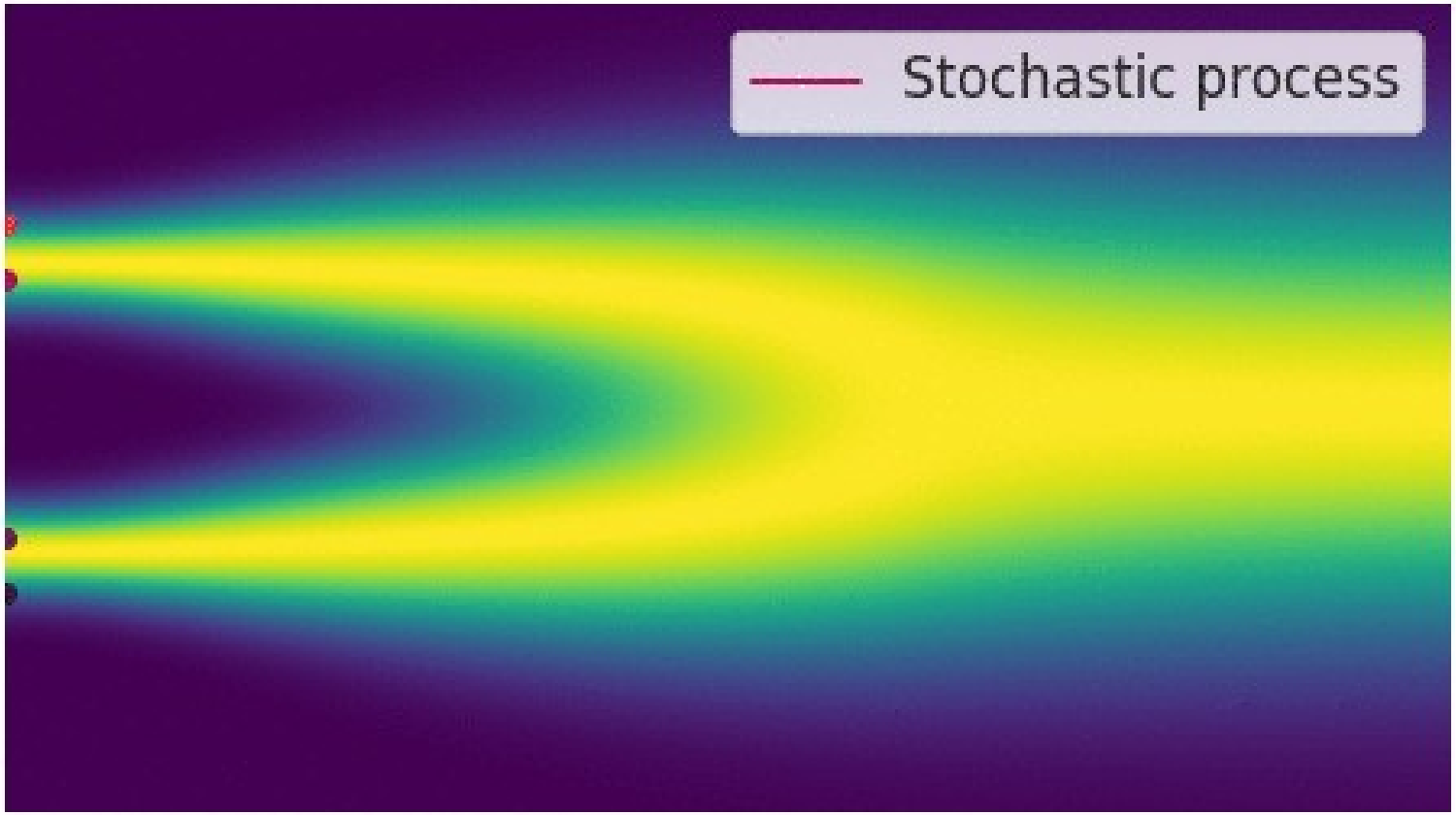
[around 2021 – started to “win”]

Still used a lot – e.g. as dim. reduction for diffusion (Latent diffusion)

LLMs are different – they are fundamentally discrete and autoregressive, modelling: p(next token | previous tokens)

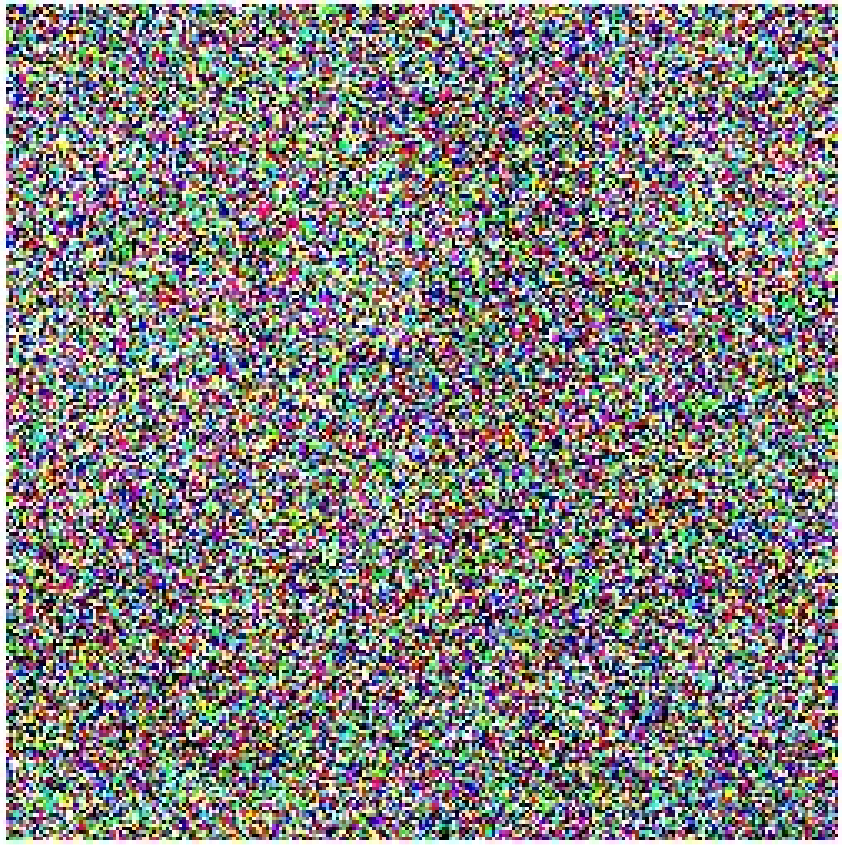
Denoising Diffusion: noisy dynamics bridging distributions

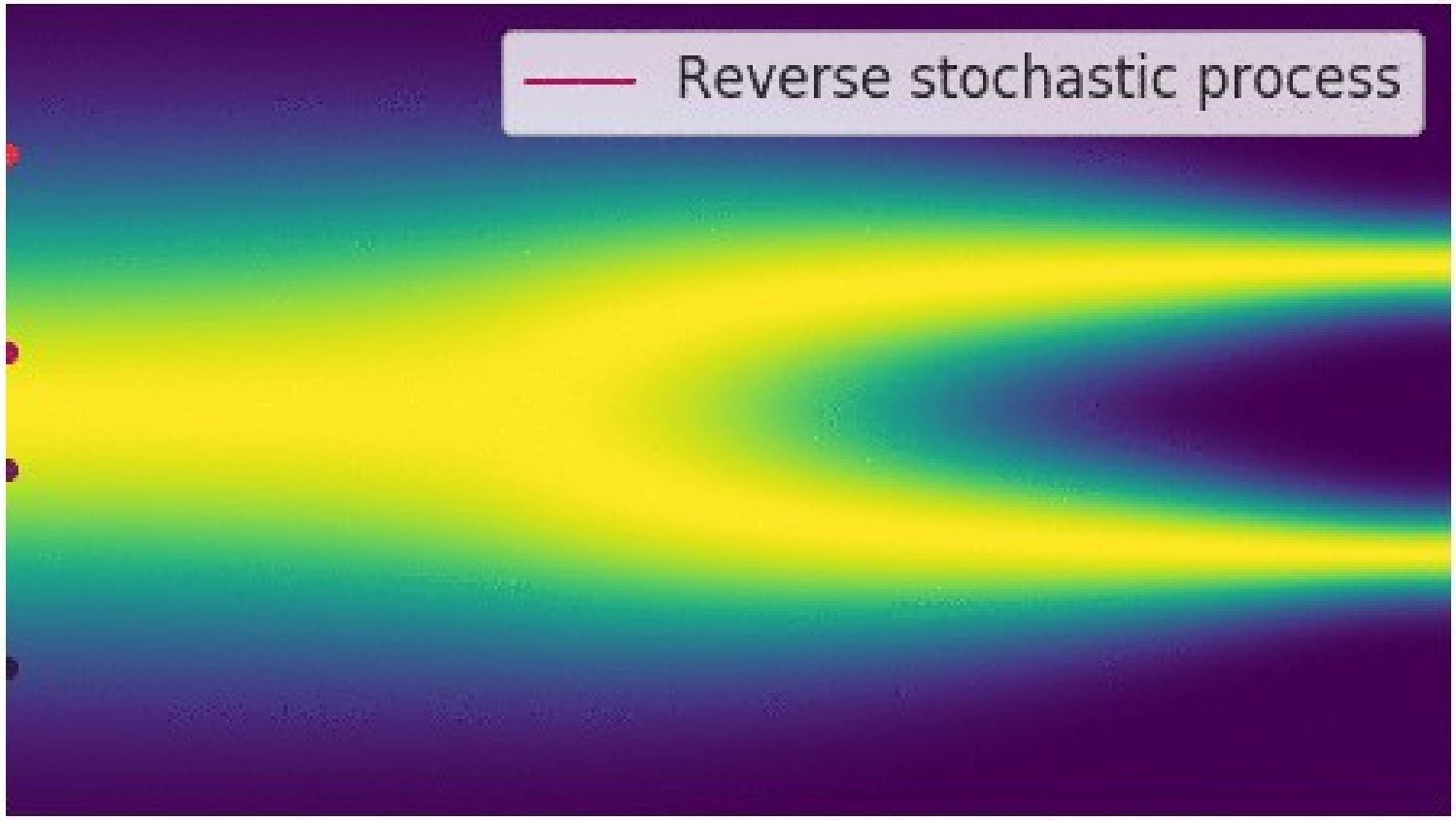




From a highly recommended intro: https://yang-song.net/blog/2021/score/

Denoising Diffusion: noisy dynamics bridging distributions





From a highly recommended intro: https://yang-song.net/blog/2021/score/

Different ways to understand diffusion models

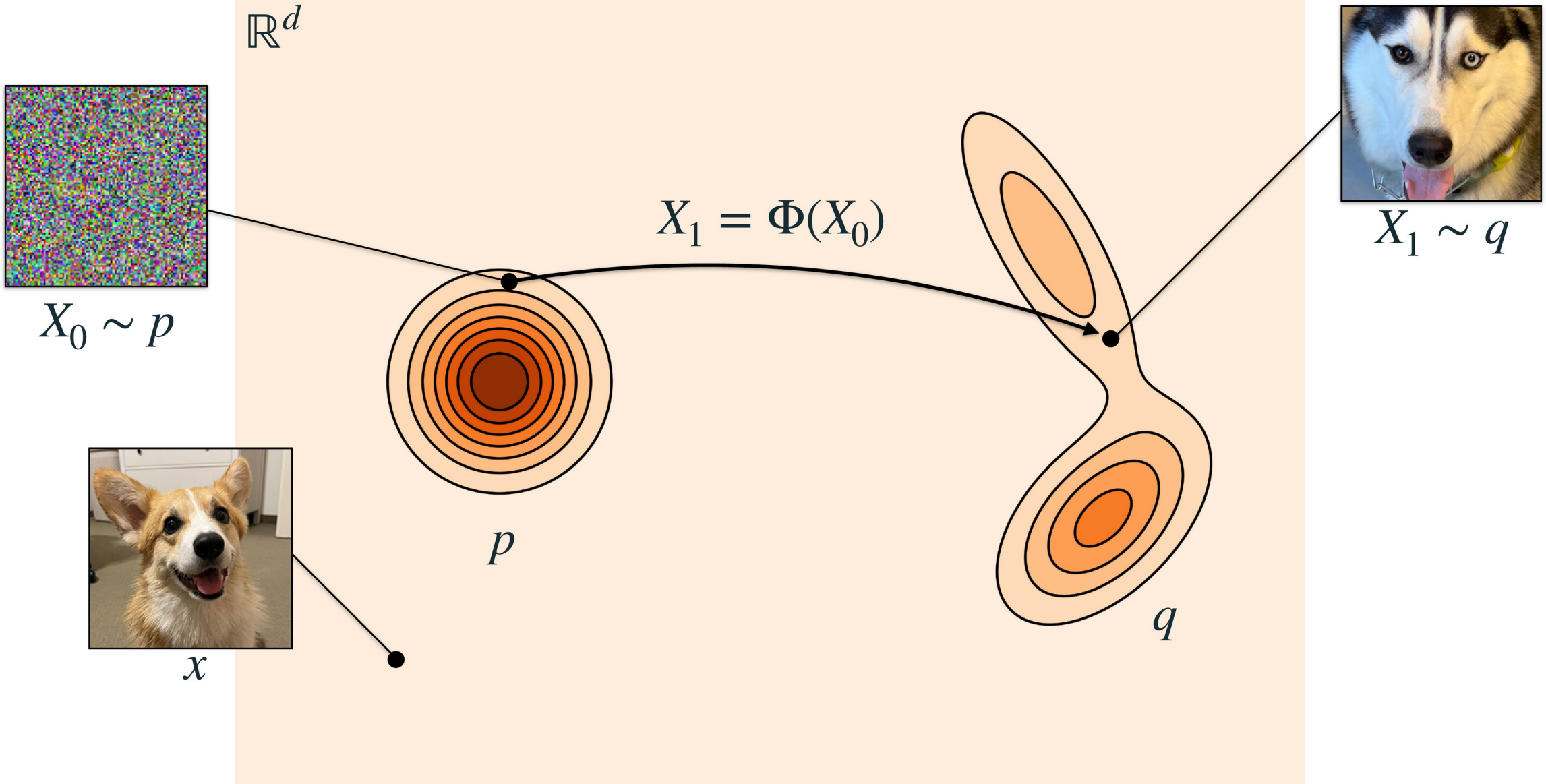
- Engineering / Denoising (Cold diffusion paper, e.g)
- Variational bound / VAE (DDPM, Ho et al)
- Score matching / Energy / sampling (Yang Song-ian perspective)
- **Flow matching / Ordinary Differential Equations (today)**
- SDEs
- Nonequilibrium dynamics dynamics / annealed importance sampling (Sohl-Dickstein)
- Information-theoretic Diffusion (my group!)
- Stochastic interpolants
- Optimal Transport

The Generative Modeling Problem

$\mathbb{R}^d$

x

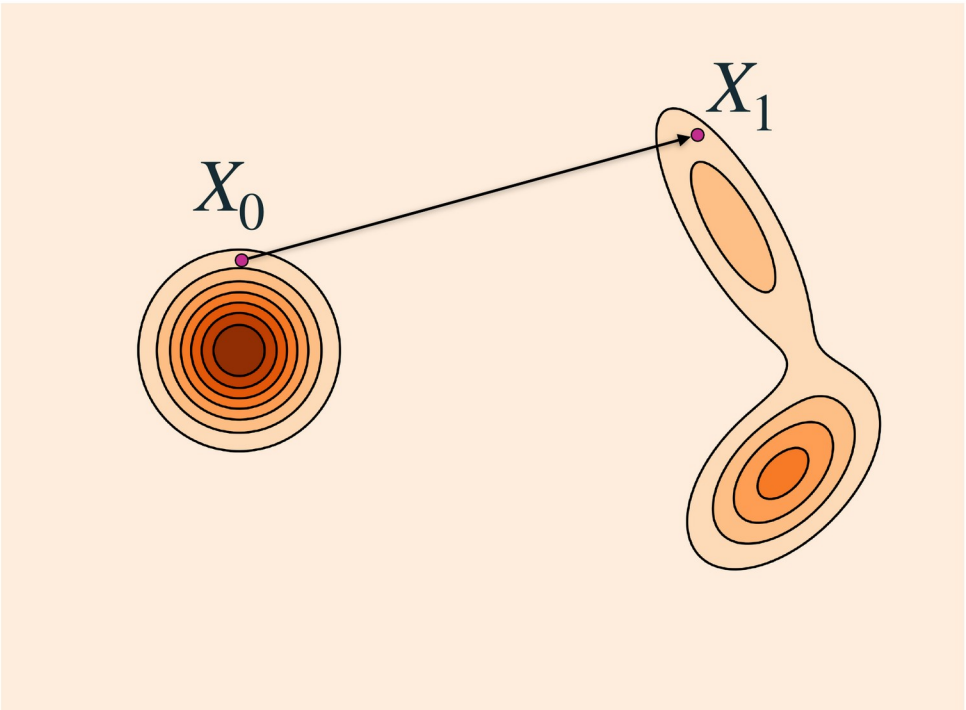
The Generative Modeling Problem



Model

- Direct map

$$X_1 = \Phi(X_0)$$



- **Pros**: efficient sampling
- **Cons**: not probabilistic
- **Cons**: min-max loss

enerative modeling via drifting

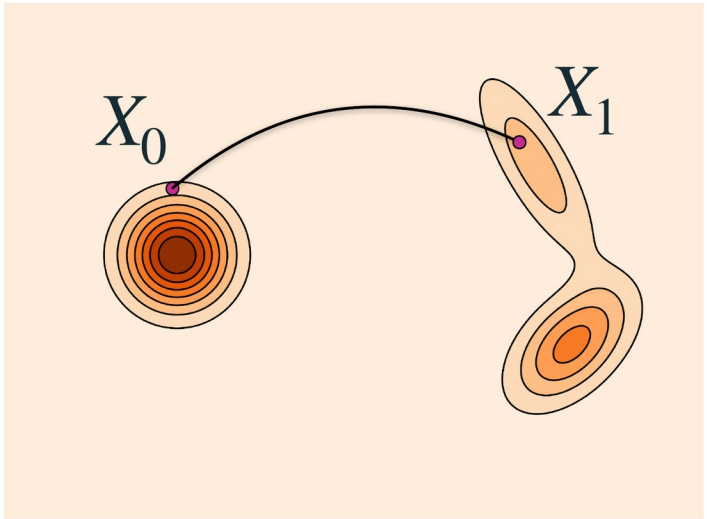
eanFlow 1 step gen modeling

"Generative Adversarial Networks" Goodfellow et al. (2014)

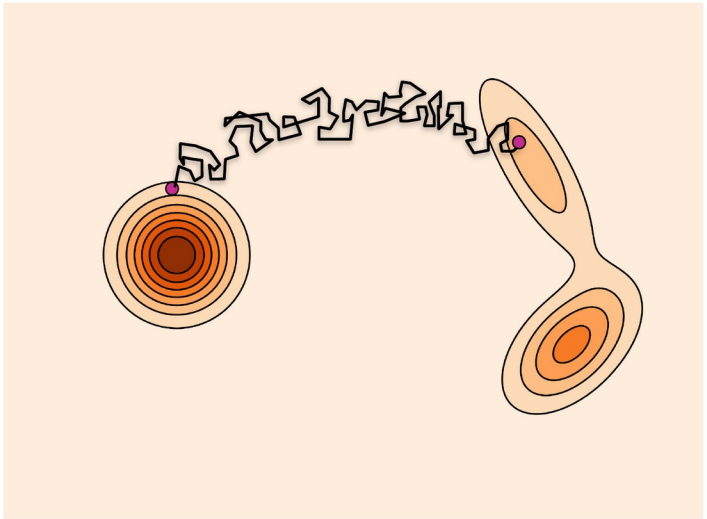
Model

- Continuous-time Markov process $(X_t)_{0 \le t \le 1}$

$$X_{t+h} \leftarrow \Phi_{t+h|t}(X_t)$$



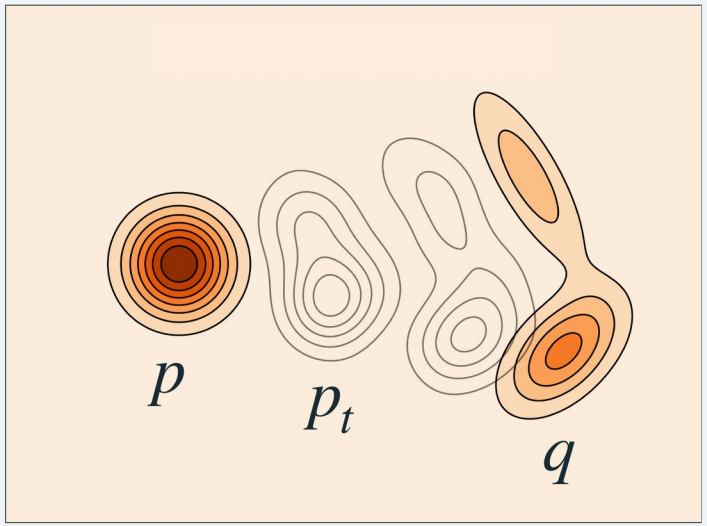
Flow

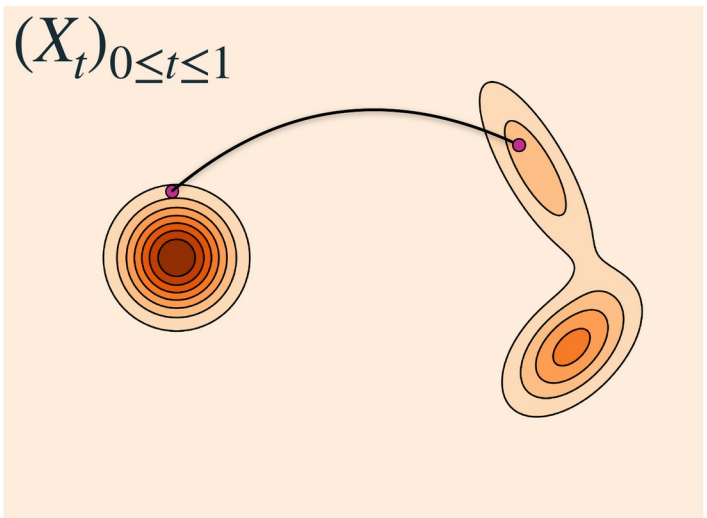


Diffusion

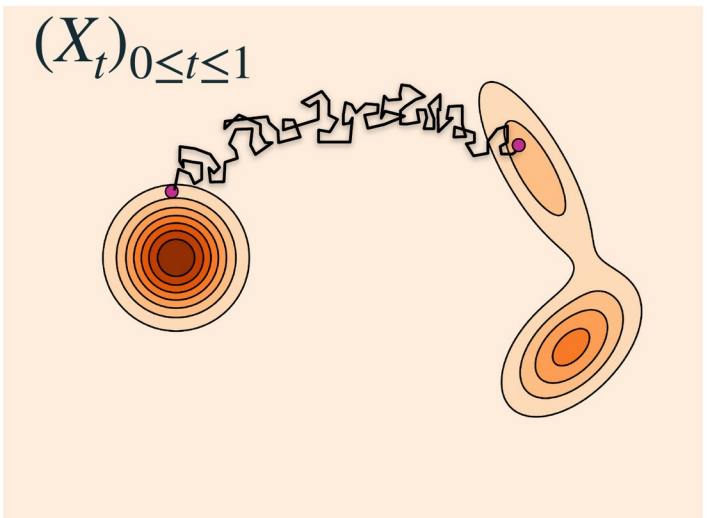
Marginal probability path

$$X_t \sim p_t$$



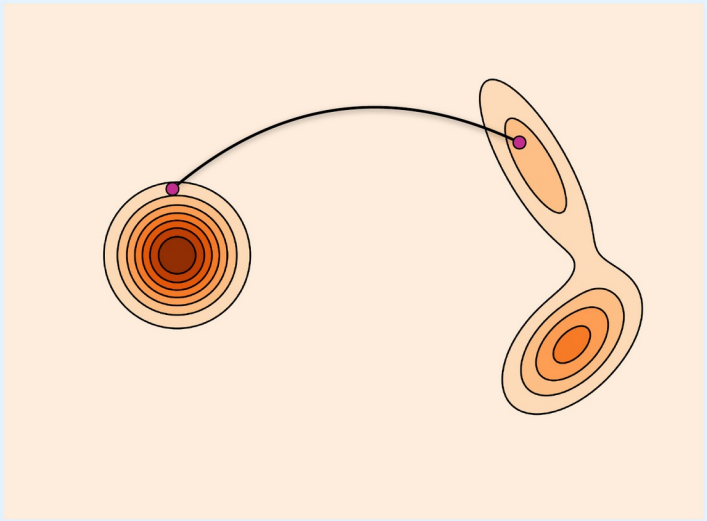


Flow



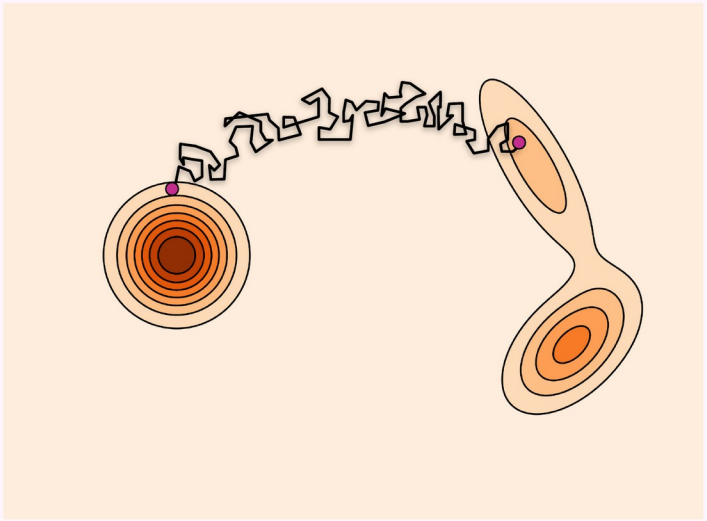
Diffusion

- For now, we focus on flows…



Flow

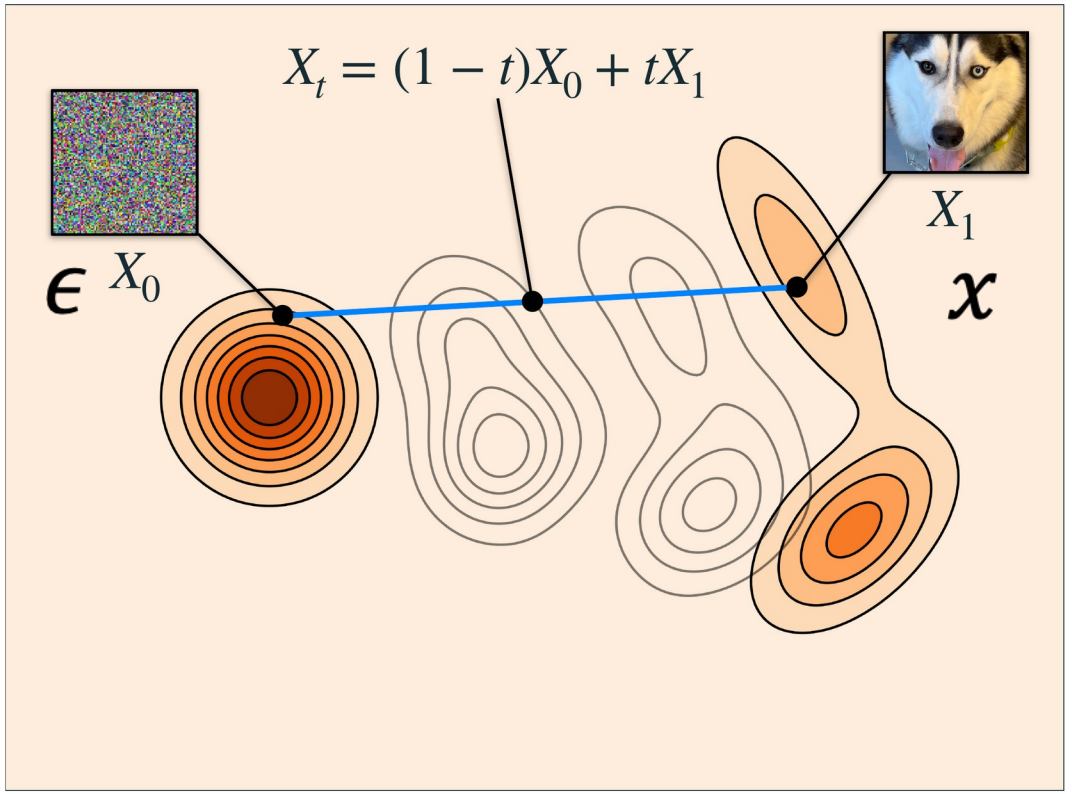
- Simple
- Faster sampling
- Exact likelihood estimator
- Flexible, easier to build



Diffusion

- Larger design space
- Slower sampling
- ELBO

$$z_t = (1-t)\epsilon + t\,x$$



Flow matching from interpolation

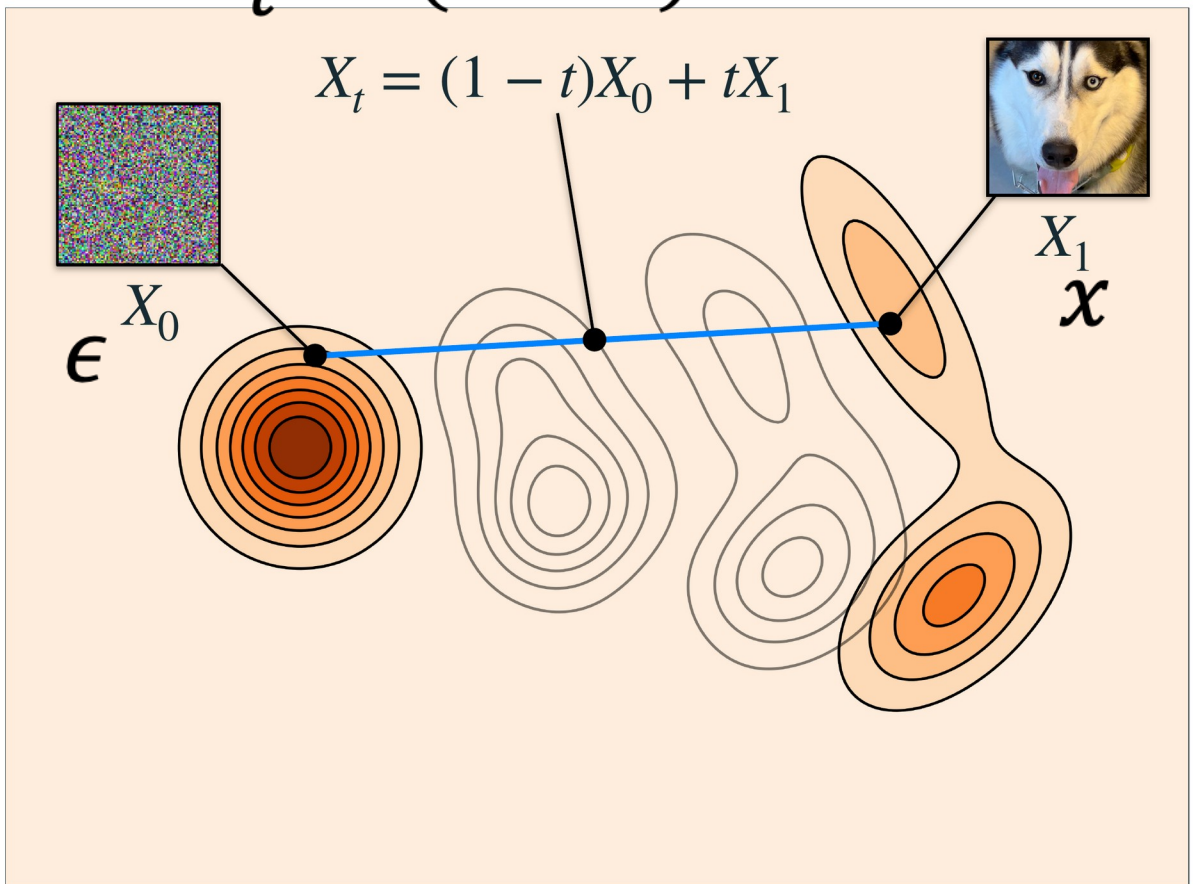
On the board

Using notation from https://arxiv.org/pdf/2511.13720

Simplest version of Flow Matching

$$z_t = (1-t)\epsilon + t\,x$$

```
for _ in range(10000):
  x_1  = Tensor(make_moons(256, noise=0.05)[0])
  x_0  = torch.randn_like(x_1)
  t    = torch.rand(len(x_1), 1)
  x_t  = (1 - t) * x_0 + t * x_1
  dx_t = x_1 - x_0
  optimizer.zero_grad()
  loss_fn(flow(x_t, t), dx_t).backward()
  optimizer.step()
```



Loss

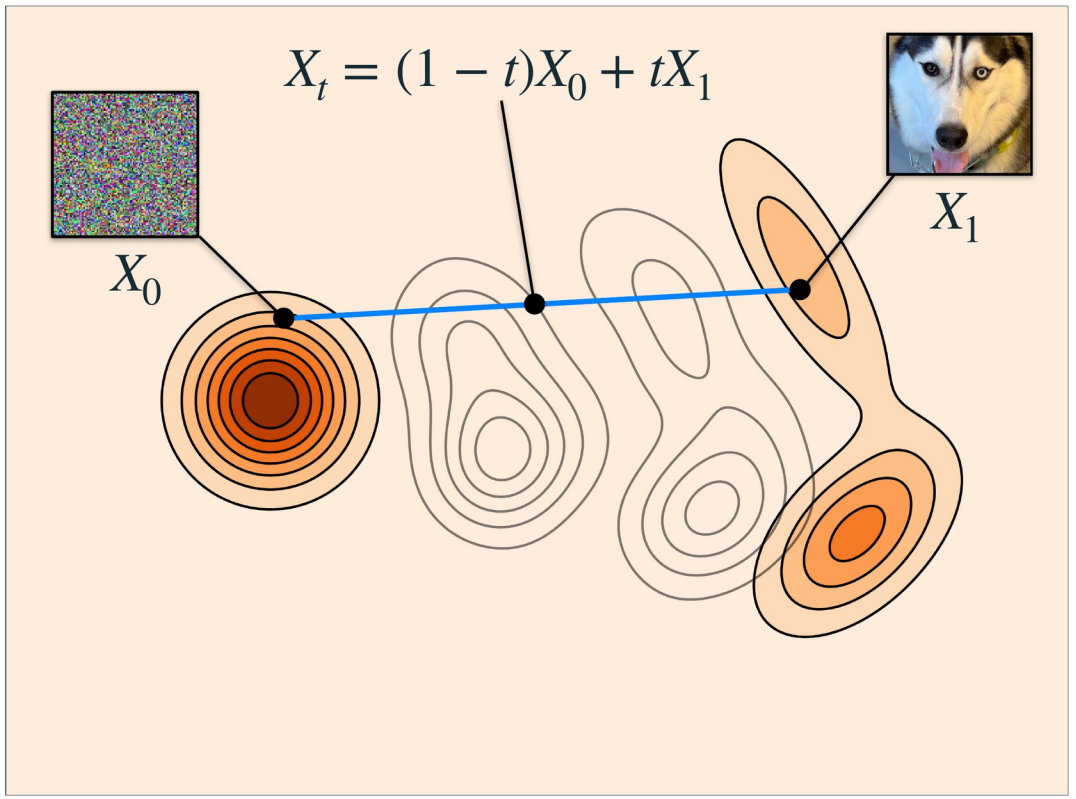
$$\mathbb{E}_{t,\epsilon,x}\left[\|v_\theta(z,t) - v\|\text{^}2\right]$$

Simplest version of Flow Matching

- Arbitrary $X_0 \sim p, X_1 \sim q$
- Arbitrary coupling $(X_0, X_1) \sim \pi_{0,1}$

Why does it work?

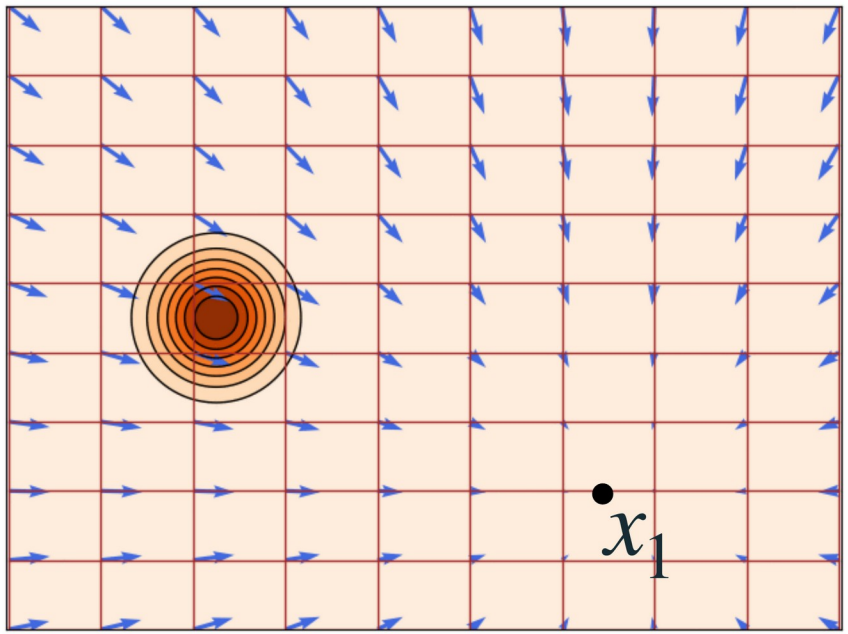
- Build flow from conditional flows
- Regress conditional flows



$$\mathbb{E}_{t,\epsilon,x}\left[\|v_\theta(z,t) - v\|\text{^}2\right]$$

Build flow from conditional flows

Generate a single target point



$$X_t = \psi_t(X_0 \mid x_1) = (1-t)X_0 + tx_1$$

$p_{t|1}(x \mid x_1)$ conditional probability

$u_t(x \mid x_1)$ conditional velocity

Starting from Continuous Normalizing Flows

Diffusion models can be seen as a continuous flow (See e.g. Karras et al paper from readings)

The multivariate change of variables formula

$$x \in \mathbb{R}^n, z = f(x)$$

$$f : \mathbb{R}^n \to \mathbb{R}^n, (\text{invertible})$$

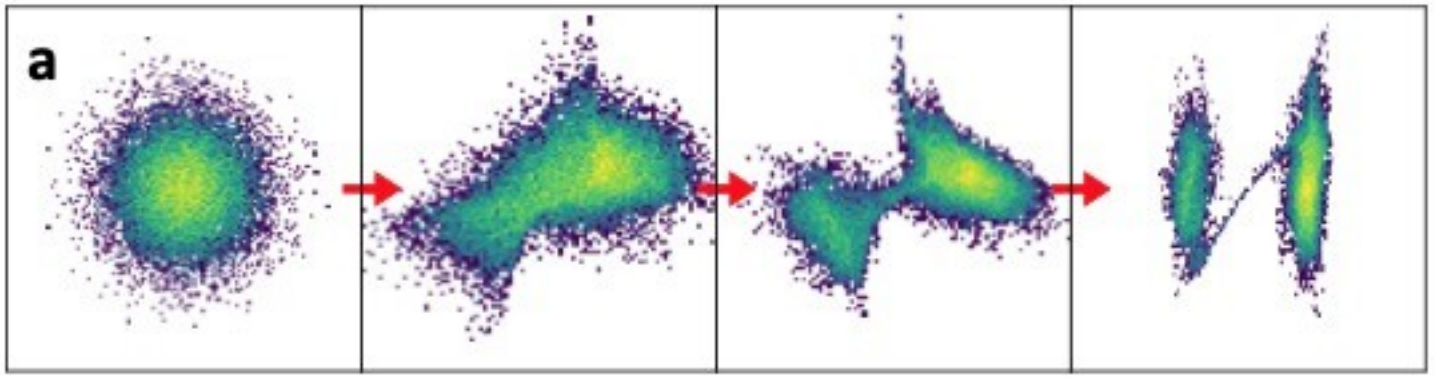
$$J_f(x) = \frac{\partial f(x)}{\partial x}$$

$$p_X(x) = p_Z(f(x)) \left|\det J_f(x)\right|$$

Challenges

$$\max_{\theta} \mathbb{E}_{x \sim \text{data}} \left[\log p_X(x) = \log p_Z(f(x)) + \log \left| \det J_f(x) \right| \right]$$

- Define a *powerful* transformation, z = f(x) (with neural nets somehow)
  - Ensure f is invertible (for change of variable formula to hold)
  - Tractably calculate the inverse $x = f^{-1}(z)$ (for sampling)
  - Tractably calculate the Jacobian (O(D^3) is the brute force!)



Transformation via continuous normalizing flows / ODE

$$\dot{z} = f(z)$$

$$z(0) = x$$

- Define by dynamics
- Invertible (just reverse direction of time to get inverse)
- *Bonus: Change of variable formula will be simpler in continuous limit!*

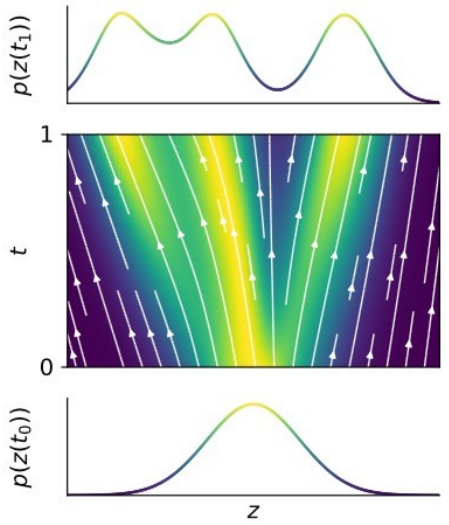


Figure 1: FFJORD transforms a simple base distribution at t_0 into the target distribution at t_1 by integrating over learned continuous dynamics.

When is the ODE invertible?

- https://en.wikipedia.org/wiki/Picard–Lindelöf_theorem

In mathematics – specifically, in differential equations – the **Picard–Lindelöf theorem**, **Picard's existence theorem**, **Cauchy–Lipschitz theorem**, or **existence and uniqueness theorem** gives a set of conditions under which an initial value problem has a unique solution.

The theorem is named after Émile Picard, Ernst Lindelöf, Rudolf Lipschitz and Augustin-Louis Cauchy.

Consider the initial value problem

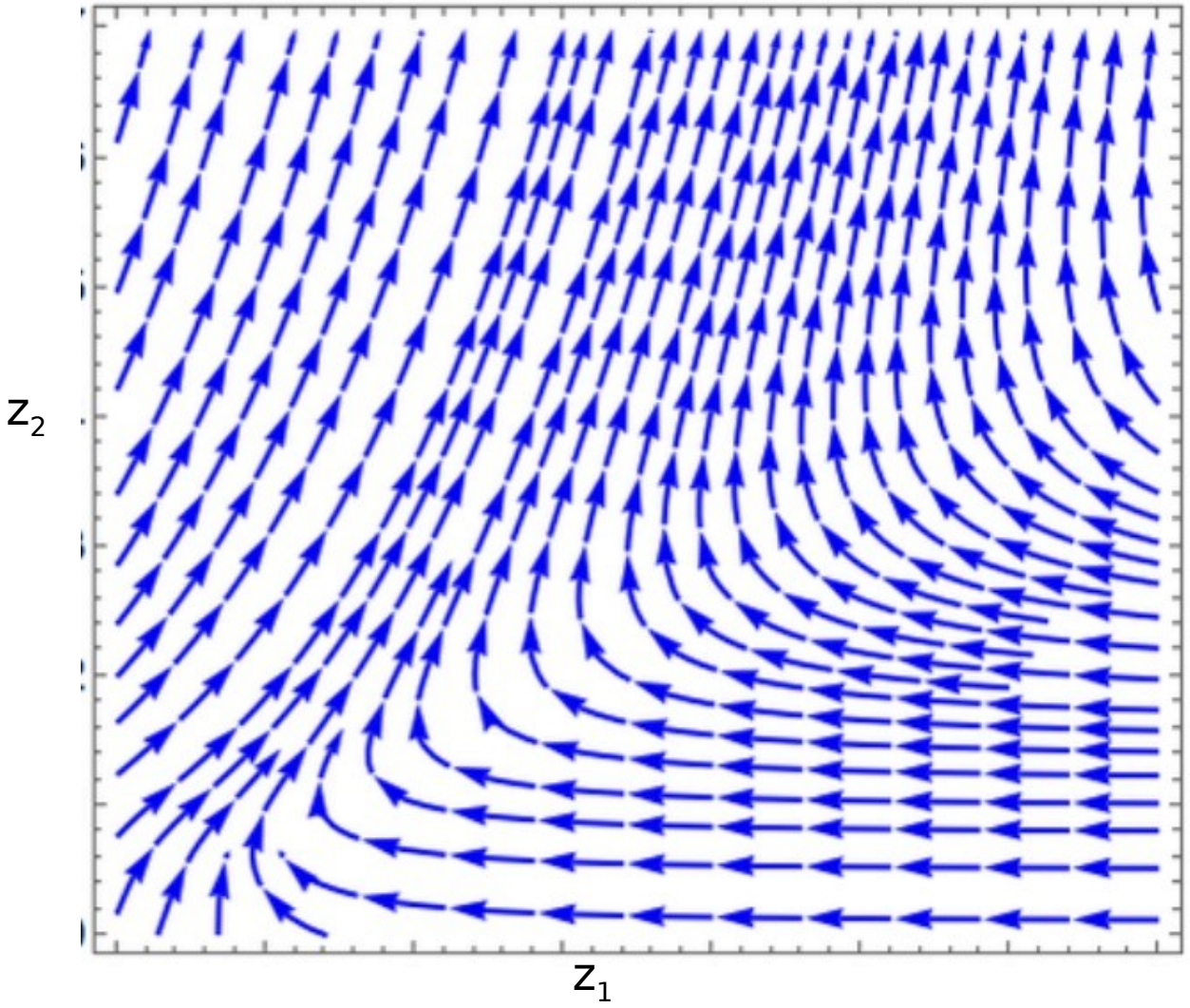
$$y'(t) = f(t, y(t)), \qquad y(t_0) = y_0.$$

Suppose f is uniformly Lipschitz continuous in y (meaning the Lipschitz constant can be taken independent of t) and continuous in t, then for some value $\varepsilon > 0$, there exists a unique solution $y(t)$ to the initial value problem on the interval $[t_0 - \varepsilon, t_0 + \varepsilon]$.[1]

How do distributions change under flows?

$$\dot{z} = f(z)$$

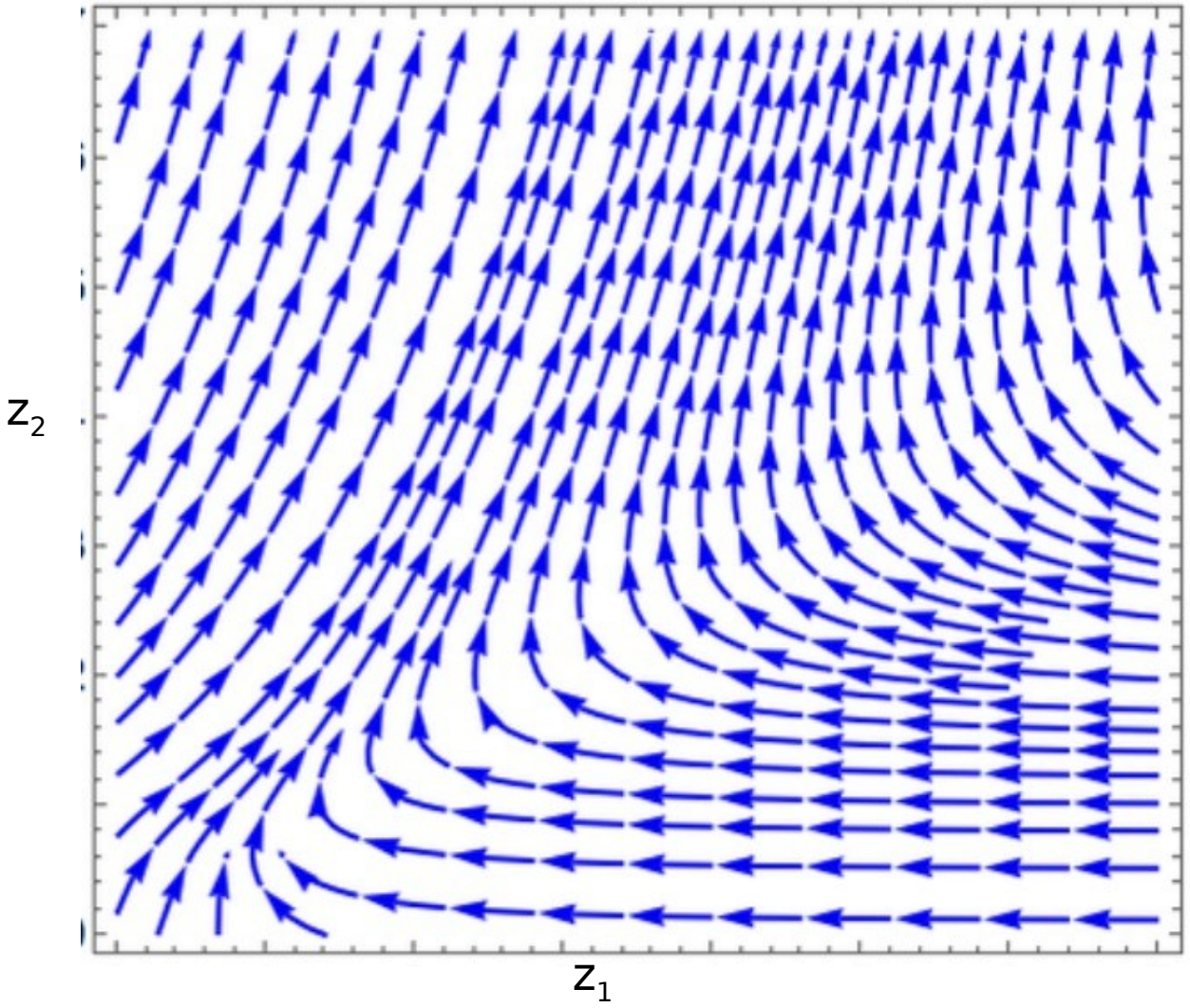
$$z(0) = x$$



How do distributions change under flows?

$$\dot{z} = f(z)$$

$$z(0) = x$$



Continuous Change of variable formula

$$\dot{z} = f(z) \rightarrow z(t+h) \approx z(t) + hf(z(t))$$

$$\partial z(t+h)/\partial(z(t)) \approx \mathbb{I} + h\partial f/\partial z$$

$$d/dt \log p(z(t), t) = \lim_{h \to 0} (\log p(z(t+h), t+h) - \log p(z(t), t)))/h$$

$$= \lim_{h \to 0} -(\log |\det \partial z(t+h)/\partial z(t)|)/h$$

- Use change of variables formula
- Log det = Tr log
- Use first order expansion of dynamics ODE
- First order expansion of log

Diffusion – reverse SDE dynamics

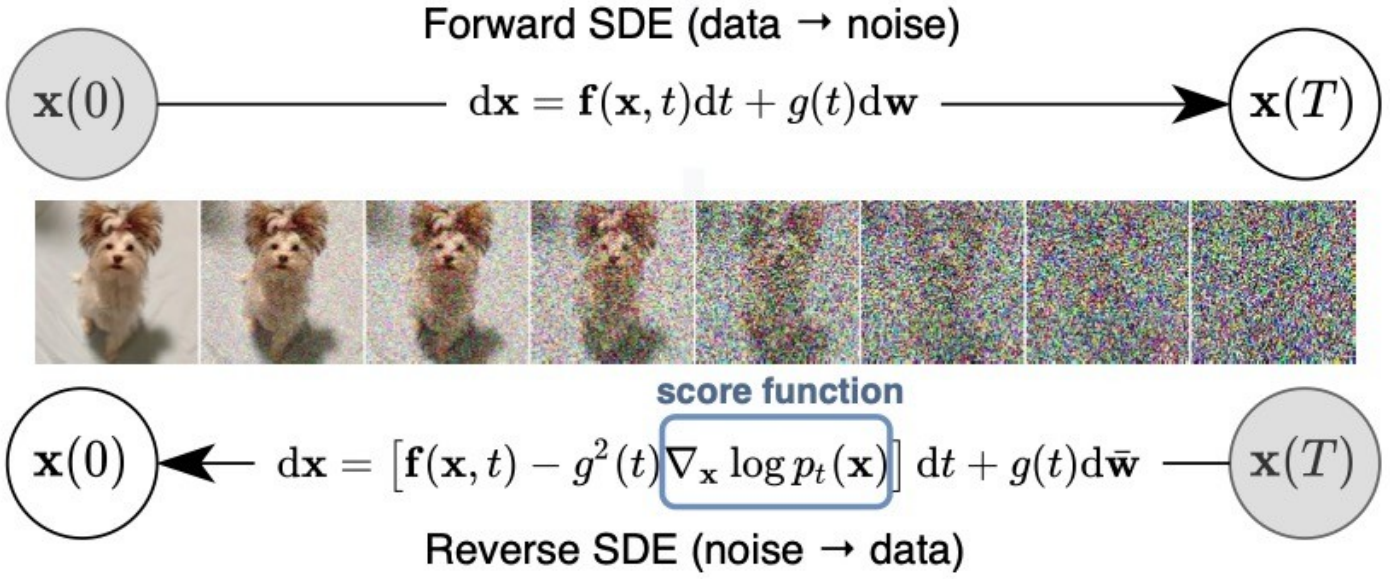
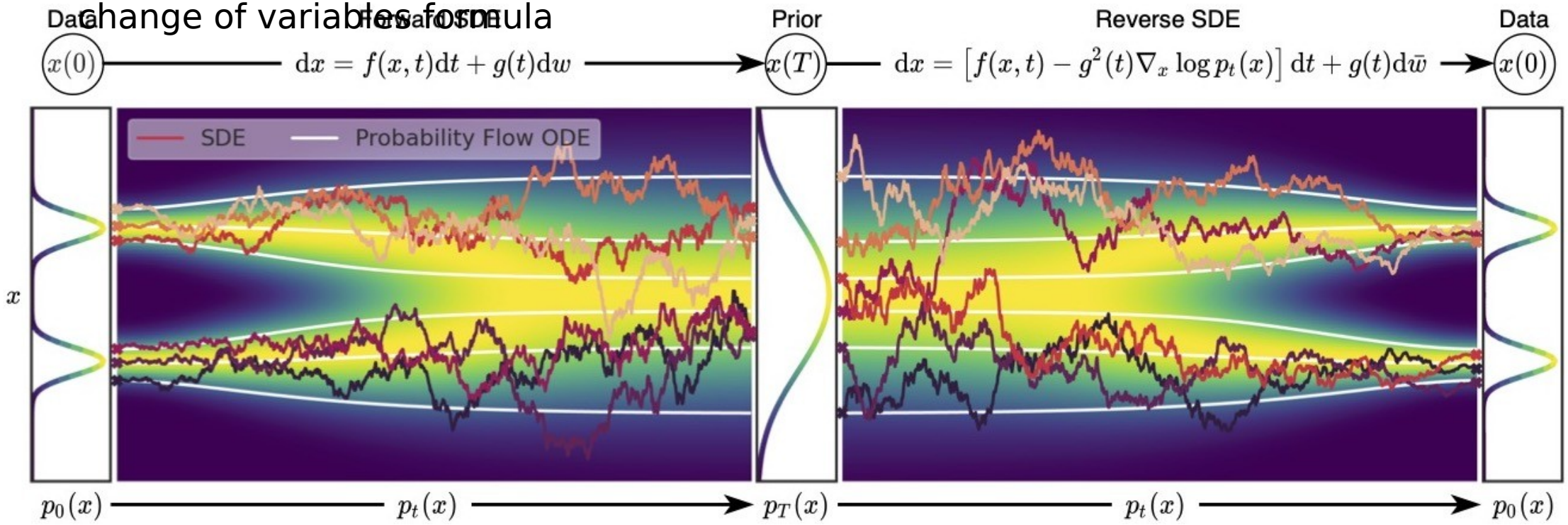


Figure 1: **Solving a reverse-time SDE yields a score-based generative model.** Transforming data to a simple noise distribution can be accomplished with a continuous-time SDE. This SDE can be reversed if we know the score of the distribution at each intermediate time step, $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$.

Convert to continuous flow?

This ODE has the same marginal densities. (Fokker Planck again!)
So if we replace the learned score function, we can solve this ODE to get probability density estimates, with the change of variables formula

$$\mathrm{d}\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\right] \mathrm{d}t.$$



Elucidating the Design Space of Diffusion-Based Generative Models

Tero Karras NVIDIA
Miika Aittala NVIDIA
Timo Aila NVIDIA
Samuli Laine NVIDIA

Abstract

We argue that the theory and practice of diffusion-based generative models are currently unnecessarily convoluted and seek to remedy the situation by presenting a design space that clearly separates the concrete design choices. This lets us identify several changes to both the sampling and training processes, as well as preconditioning of the score networks. Together, our improvements yield new state-of-the-art FID of 1.79 for CIFAR-10 in a class-conditional setting and 1.97 in an unconditional setting, with much faster sampling (35 network evaluations per image) than prior designs. To further demonstrate their modular nature, we show that our design changes dramatically improve both the efficiency and quality obtainable with pre-trained score networks from previous work, including improving the FID of a previously trained ImageNet-64 model from 2.07 to near-SOTA 1.55, and after re-training with our proposed improvements to a new SOTA of 1.36.

Continuous normalizing flow – just needs the score function

$$\mathrm{d}\boldsymbol{x} = -\dot{\sigma}(t)\,\sigma(t)\,\nabla_{\boldsymbol{x}} \log p\big(\boldsymbol{x}; \sigma(t)\big)\,\mathrm{d}t,$$

Then Fokker Planck equation says that
p(x; sigma(t)) evolves from Gaussian to target distribution under these dynamics!

- $\nabla_{\boldsymbol{x}} \log p\big(\boldsymbol{x}; \sigma(t)\big)$

Continuous normalizing flow – just needs the score function

$$\mathrm{d}\boldsymbol{x} = -\dot{\sigma}(t)\, \sigma(t)\, \nabla_{\boldsymbol{x}} \log p\big(\boldsymbol{x}; \sigma(t)\big)\, \mathrm{d}t,$$

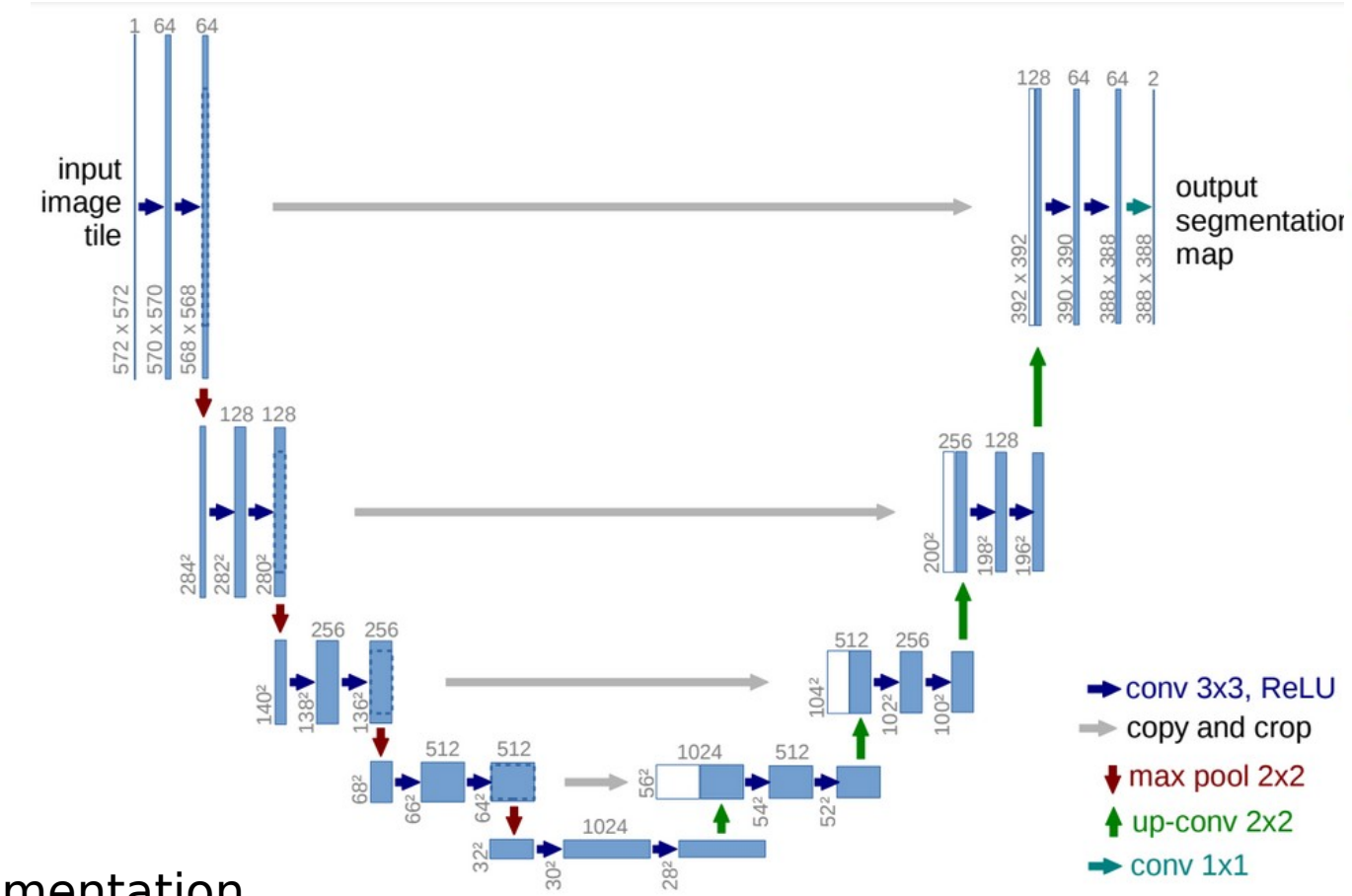
Then Fokker Planck equation says that
p(x; sigma(t)) evolves from Gaussian to target distribution under these dynamics!

Architectures and parametrization

ADD x, eps, v prediction

U-Net, the Image-to-Image Architecture

Input



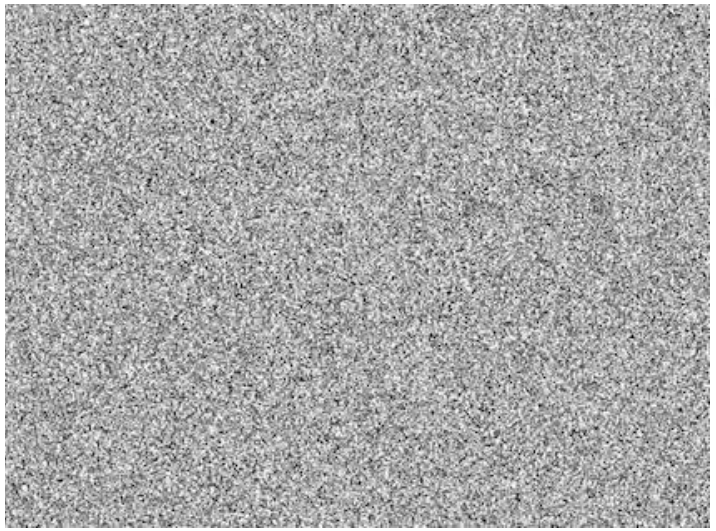
Prediction

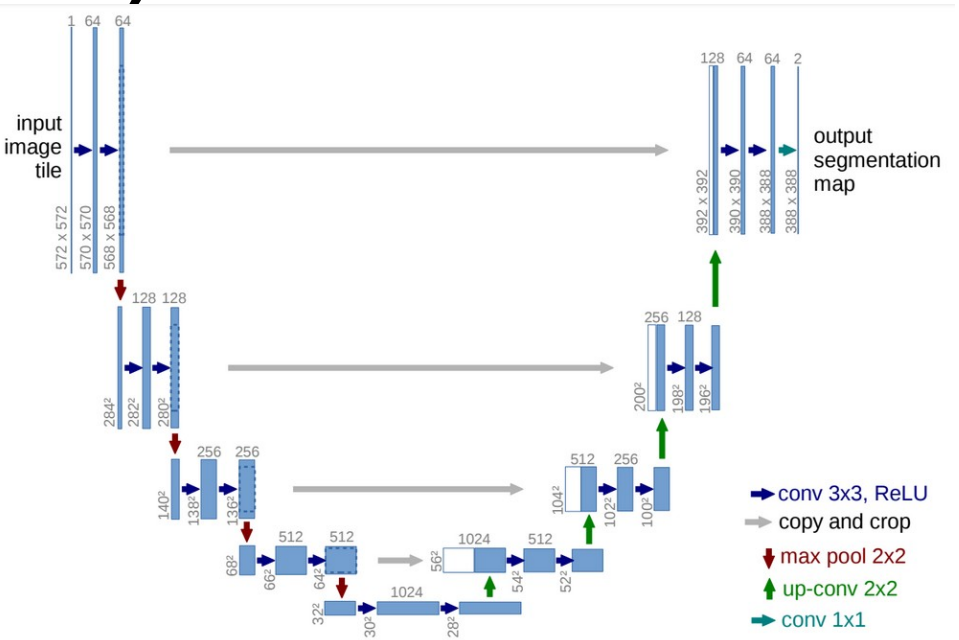
Ground truth

ginally developed for segmentation
w variations add some self-attention
ry powerful for many tasks...

Deep Image Prior (Ulyanov et al, CVPR 2018)

z, a fixed, random image





f(z), output image image

(f) U-net, depth=5

Trained with only this one data point!

(a) Input (white=masked)

It doesn't know how to fill in the blank spots. The architectural bias matches well with our perception

Zoom in

(a) Input (white=masked)

(f) U-net, depth=5

The Vision Transformer

Published as a conference paper at ICLR 2021

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy[*,†], **Lucas Beyer**[*], **Alexander Kolesnikov**[*], **Dirk Weissenborn**[*], **Xiaohua Zhai**[*], **Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby**[*,†]

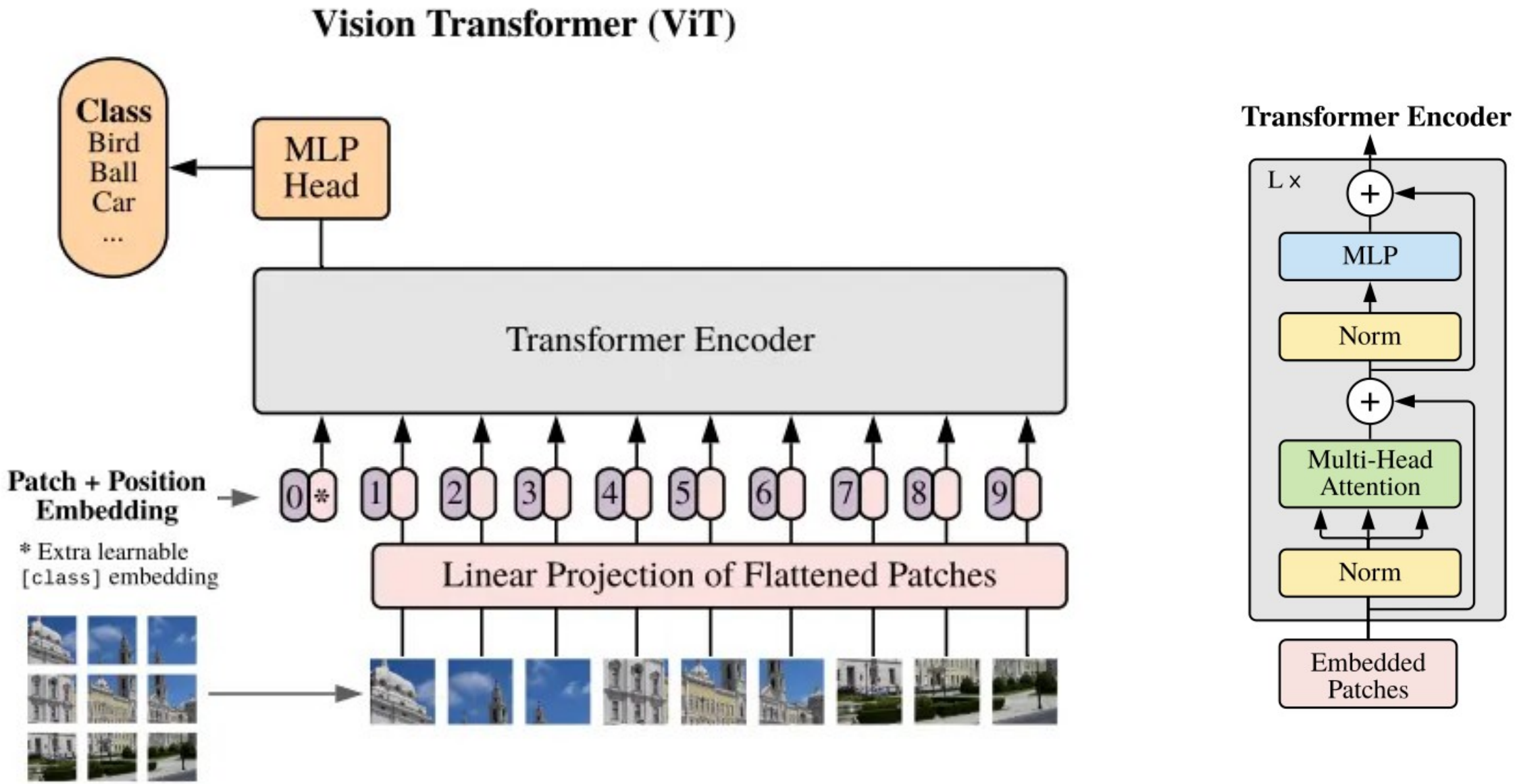
[*]equal technical contribution, [†]equal advising

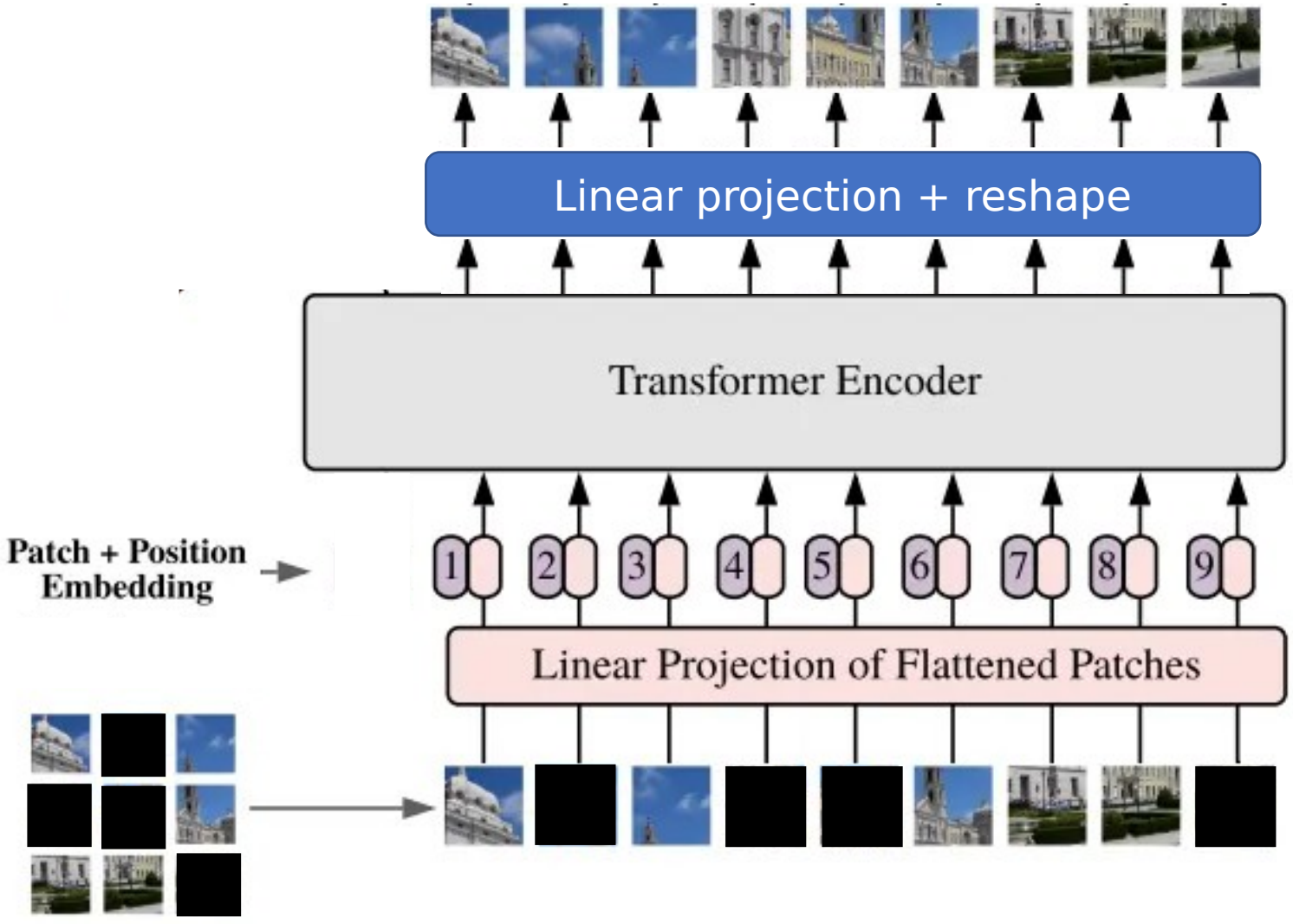
Google Research, Brain Team

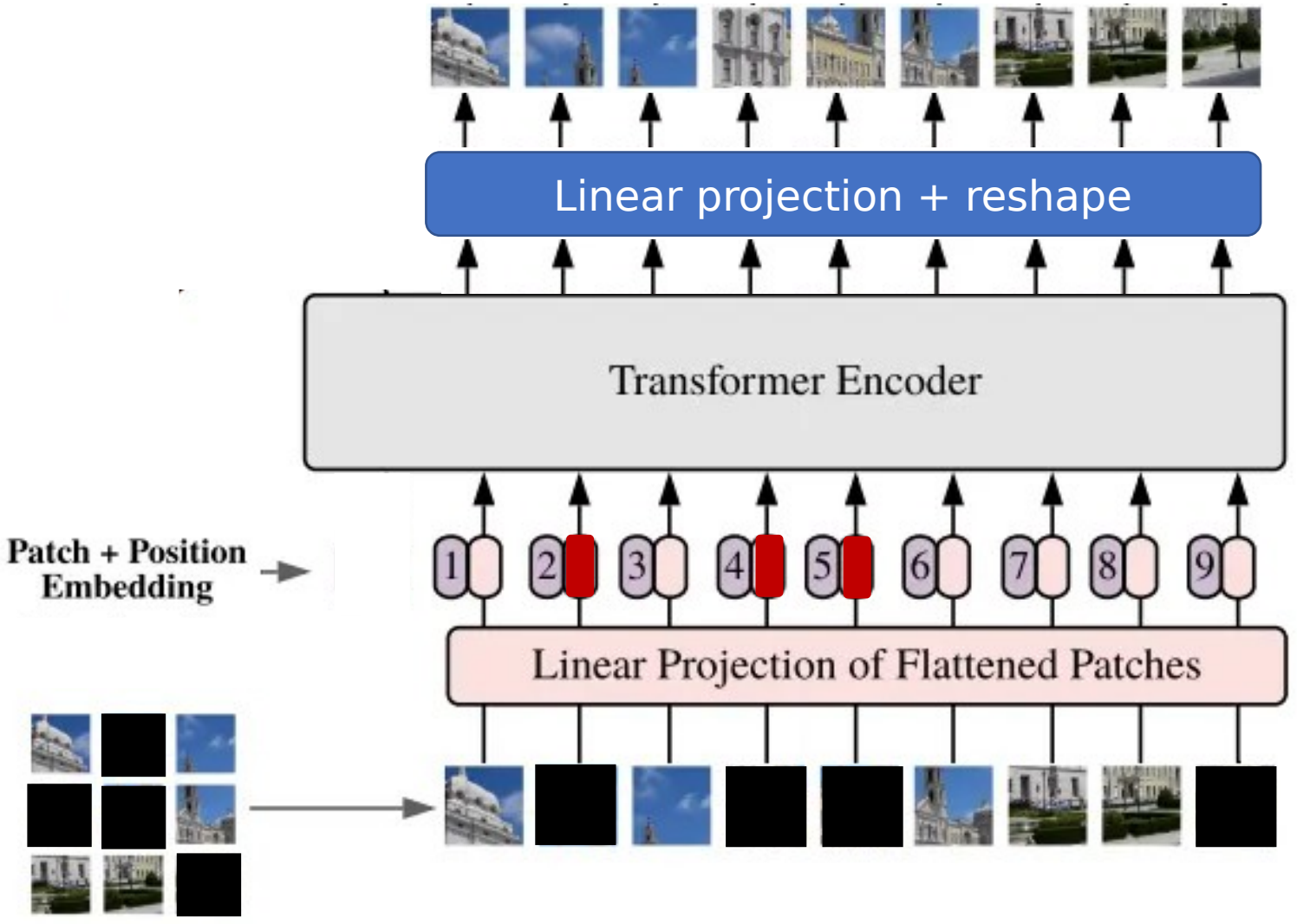
{adosovitskiy, neilhoulsby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.[1]


Vision Transformer (ViT)
Class
Bird
Ball
Car
...
MLP Head
Transformer Encoder
Patch + Position Embedding
* Extra learnable [class] embedding
0 * 1 2 3 4 5 6 7 8 9
Linear Projection of Flattened Patches
Transformer Encoder
L x
MLP
Norm
Multi-Head Attention
Norm
Embedded Patches


Linear projection + reshape
Transformer Encoder
Patch + Position Embedding
1
2
3
4
5
6
7
8
9
Linear Projection of Flattened Patches

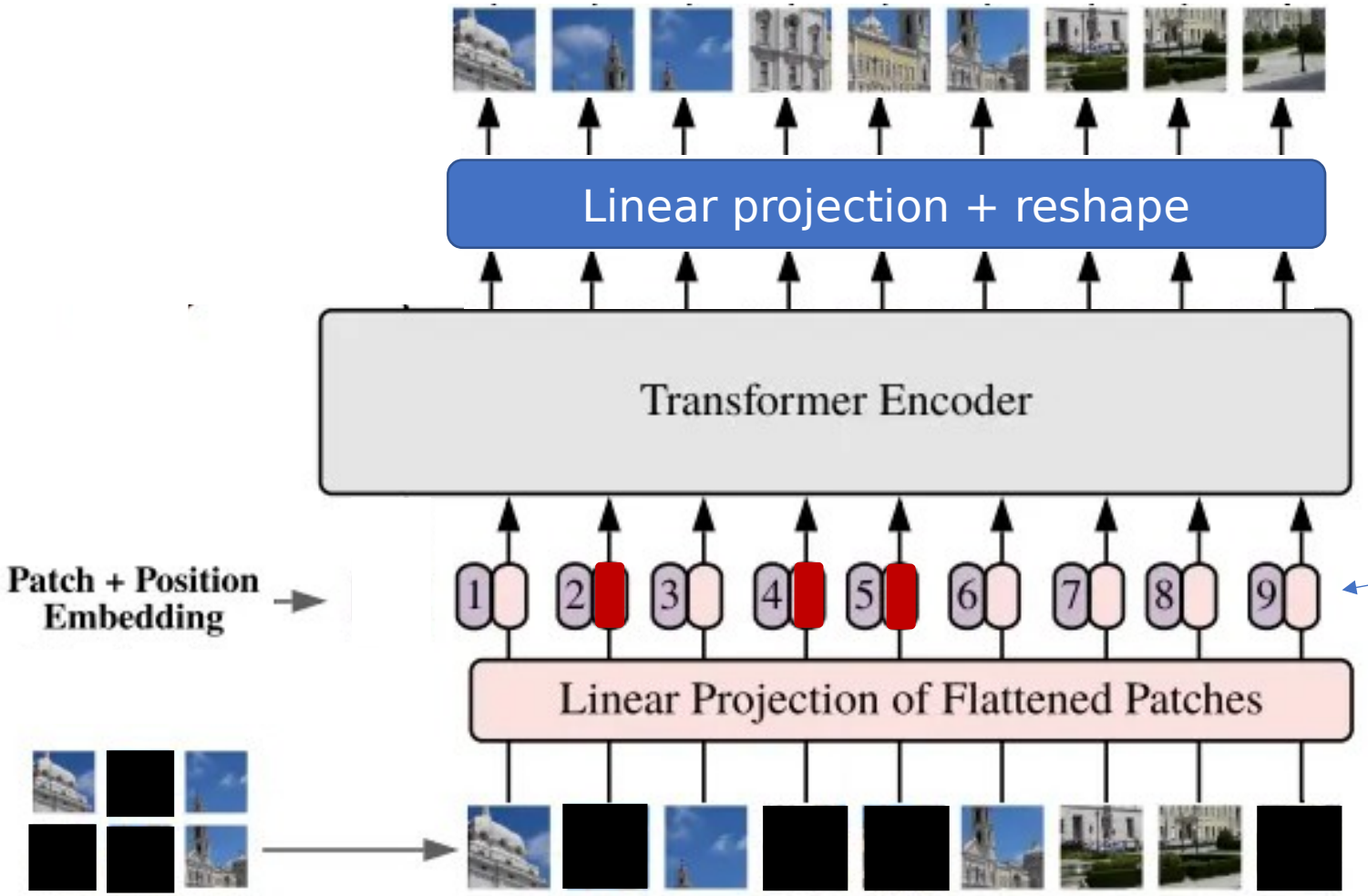


learnable Mask token

Some details we can think about later

- We'll work with 32 x 32 images again
- Convert to 4x4 patches
- Hence n_patches = 64, or 8 x 8 patches
- A patch, when flatted would be 4x4x 3 channels -> 48 dimensional when flattened
- Then we use a linear layer to project to embedding dimension. We'll use d_embed=128, a typical (small) choice for transformer

- The position embedding would be a learnable tensor, n_patches by d_embed



d_embed = 128

N_patches

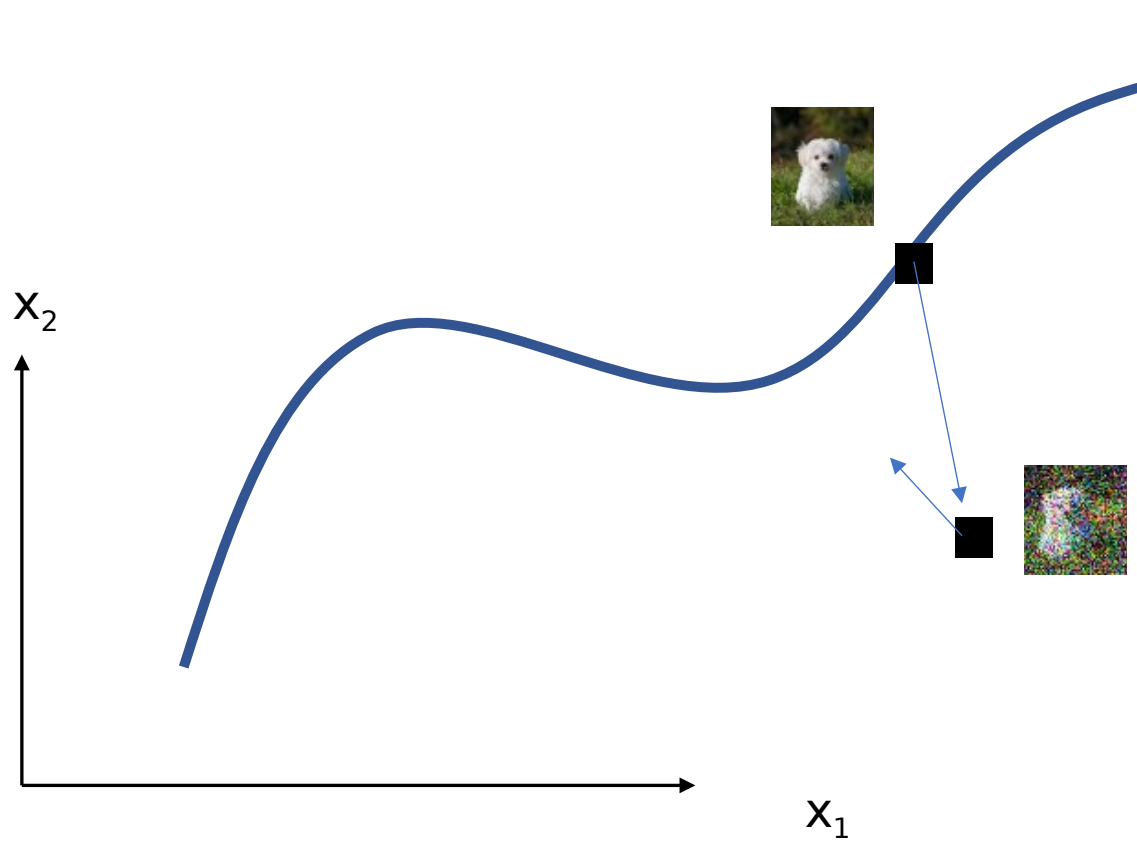
3x4x4 patches
(channels, height, width)

3x32x32 image
(channels, height, width)

Learnable Mask token, size d_emb

Engineering / autoencoder perspective

Simple denoising is good, but not enough...



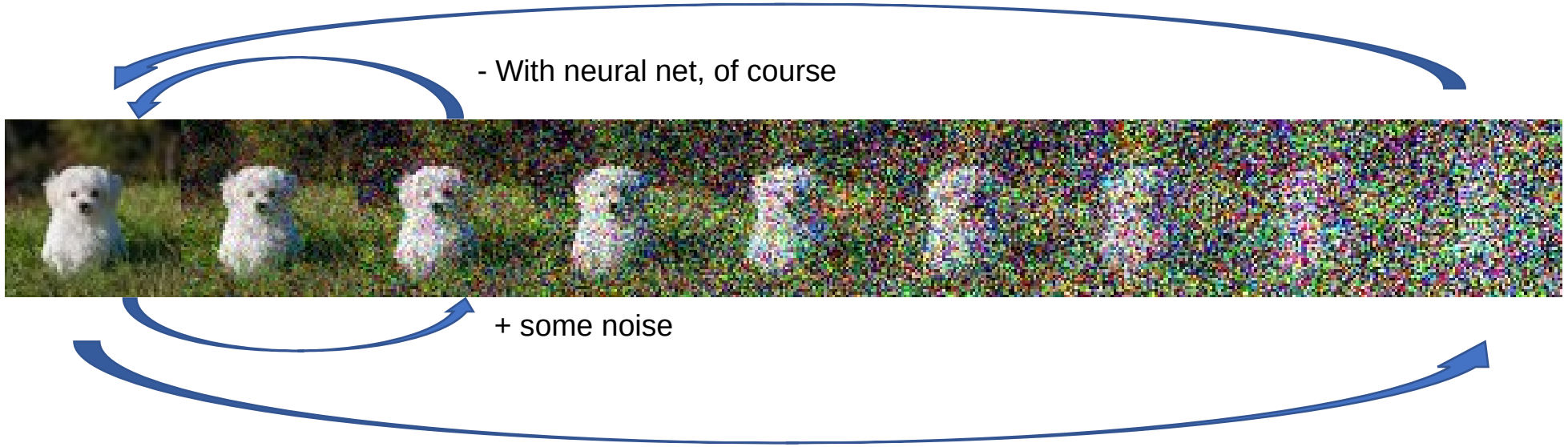
High probability manifold in data space

Issues with this picture?

High-dimensional space is *big*

Diffusion = denoising autoencoder w/ multiple noise levels

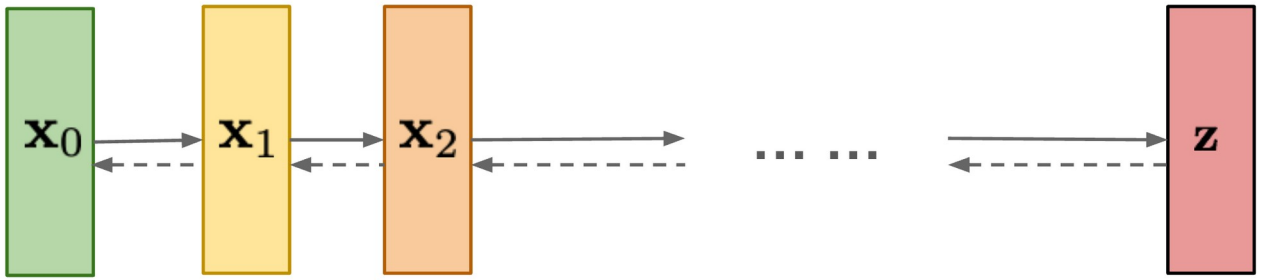
Diffusion model



When generating, we don’t have to immediately “jump” to the manifold, we can go through a sequence of intermediate less noisy steps

Representation + Objective + Optimizer = ML

Diffusion models:
Gradually add Gaussian noise and then reverse

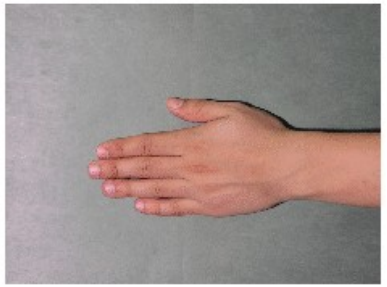


Representation/Model: U-Nets

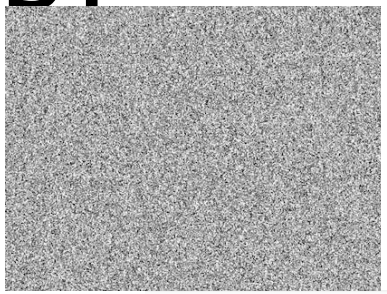
Optimizer: Adam or SGD

Loss: ?? (a few closely related versions, but basically MSE for denoising!)

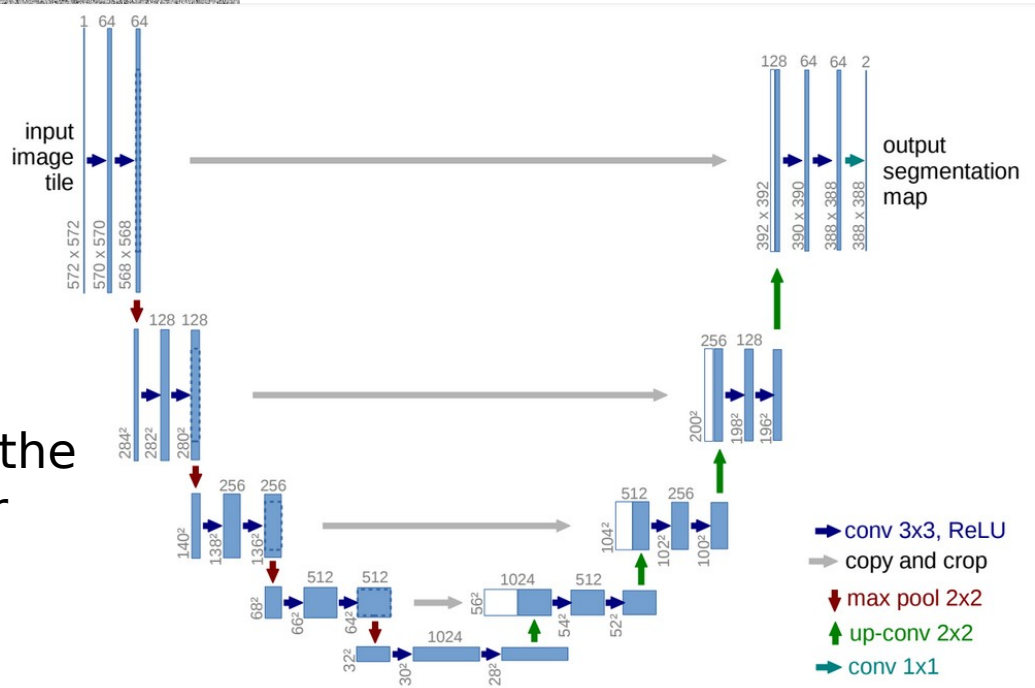
How do we use U-Nets in diffusion models?



+



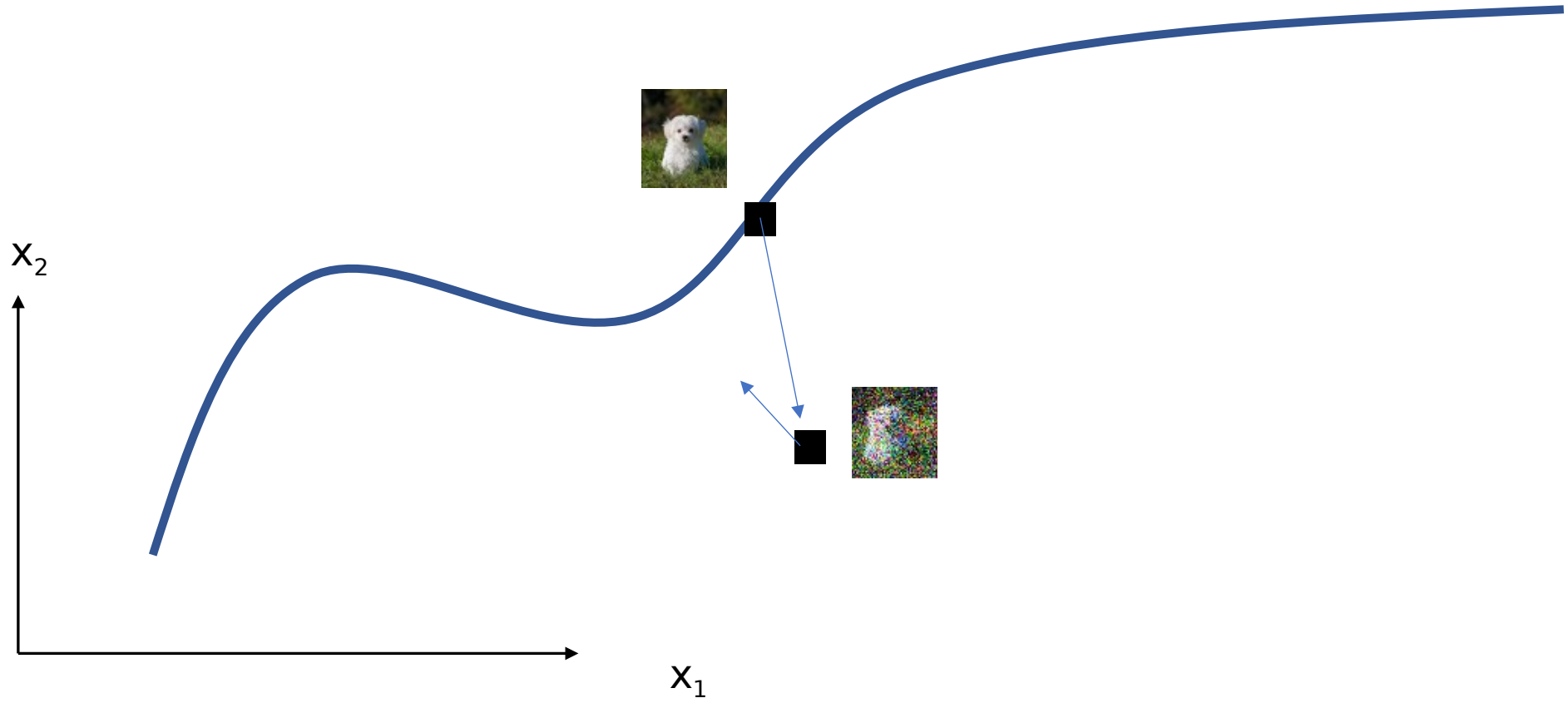
=



Observations:

- We predict the *noise,* not the signal! Well, Maybe easier because the noise distribution is Gaussian
- So to recover signal, we'll have to subtract out noise
- How much noise? *Different*

Draw



Diffusion as a variational autoencoder

One line ELBO derivation – diffusion connection?

The "gap" in the ELBO

$$\log p(x) \geq \log p(x) - D_{KL}[q(z|x)||p(z|x)]$$

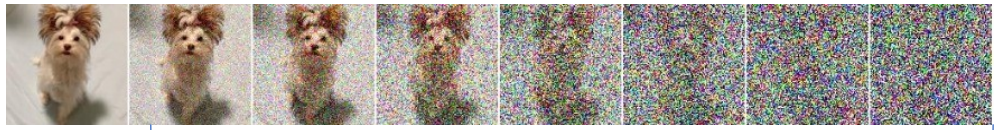
Variational approximation

$$= \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}[q(z|x)||p(z)]$$

Reconstruction

Compression/prior

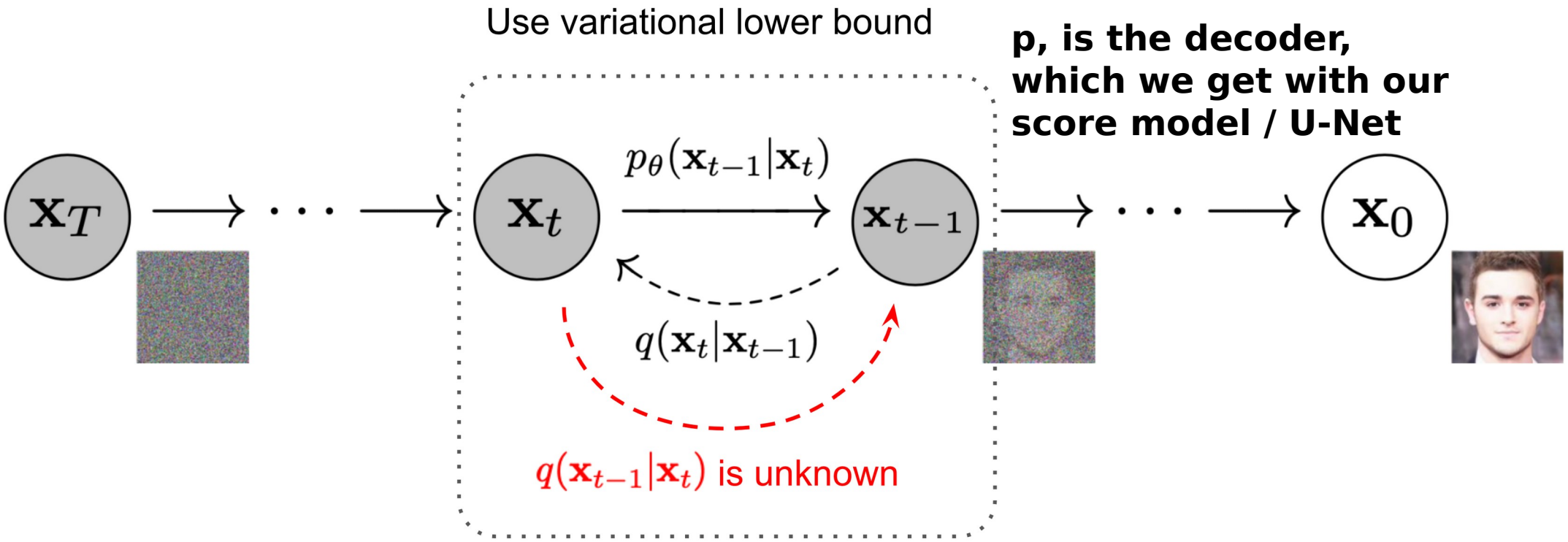
- What if "z" = a whole sequence of noisified images?



x

z

Variational formulation



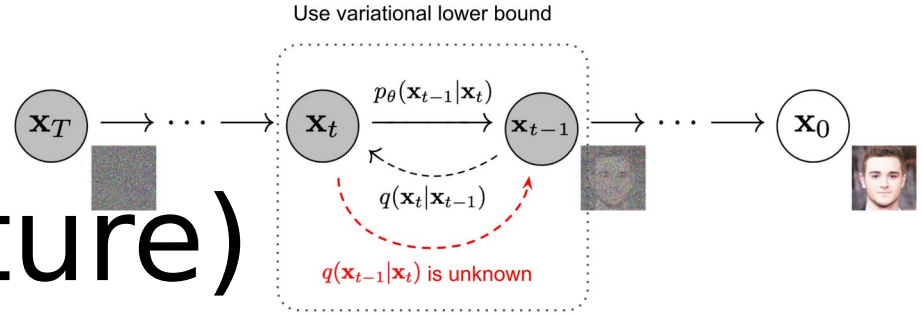
← Time-steps or amount of noise added

q, ENCODER – not learned!!
Just a sequence of Gaussian noise!

The ELBO for this weird encoder/decoder (straight from classic Ho et al 2020, DDPM paper)

$$
\begin{aligned}
-\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\mathrm{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0) \| p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T}\sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T}) / p_\theta(\mathbf{x}_0)} \right] \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} + \log p_\theta(\mathbf{x}_0) \right] \\
&= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \\
\text{Let } L_{\mathrm{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \geq -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0)
\end{aligned}
$$

Break it up (w/Markov chain structure)



$$L_{\text{VLB}} = \mathbb{E}_{q(\mathbf{x}_{0:T})}\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right]$$

$$= \mathbb{E}_q\left[\log \frac{\prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T)\prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}\right]$$

$$= \mathbb{E}_q\left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^{T}\log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}\right]$$

$$= \mathbb{E}_q\left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^{T}\log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \text{ !!} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}\right]$$

$$L_{\text{VLB}} = \mathbb{E}_{q(\mathbf{x}_{0:T})}\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right]$$

$$q(x_t|x_{t-1})$$

$$= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^{T} \log \Big(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \cdot \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} \Big) + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big]$$

$$= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^{T} \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \sum_{t=2}^{T} \log \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big]$$

$$= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^{T} \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big]$$

$$= \mathbb{E}_q\Big[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^{T} \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \Big]$$

$$= \mathbb{E}_q[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}]$$

$$
\begin{aligned}
L_{\text{VLB}} &= L_T + L_{T-1} + \cdots + L_0 \\
\text{where } L_T &= D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T)) \\
L_t &= D_{\text{KL}}(q(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1})) \text{ for } 1 \leq t \leq T-1 \\
L_0 &= -\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)
\end{aligned}
$$

The L_t terms are all Gaussian. When you simplify and parametrize in terms of our noise predictor / score matching U-Net

Ho et al find that:

$$
L_{\text{VLB}} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2 \right]
$$

Contrast: Information theoretic diffusion (ICLR 2023)

Xianghao Kong, Rob Brekelmans, Greg Ver Steeg

Relating diffusion to information/probability models

$$-\log p(\mathbf{x}) = 1/2 \int_0^\infty \mathrm{mmse}(\mathbf{x}, \gamma)\ d\gamma + c$$

Probability **Optimal denoising**

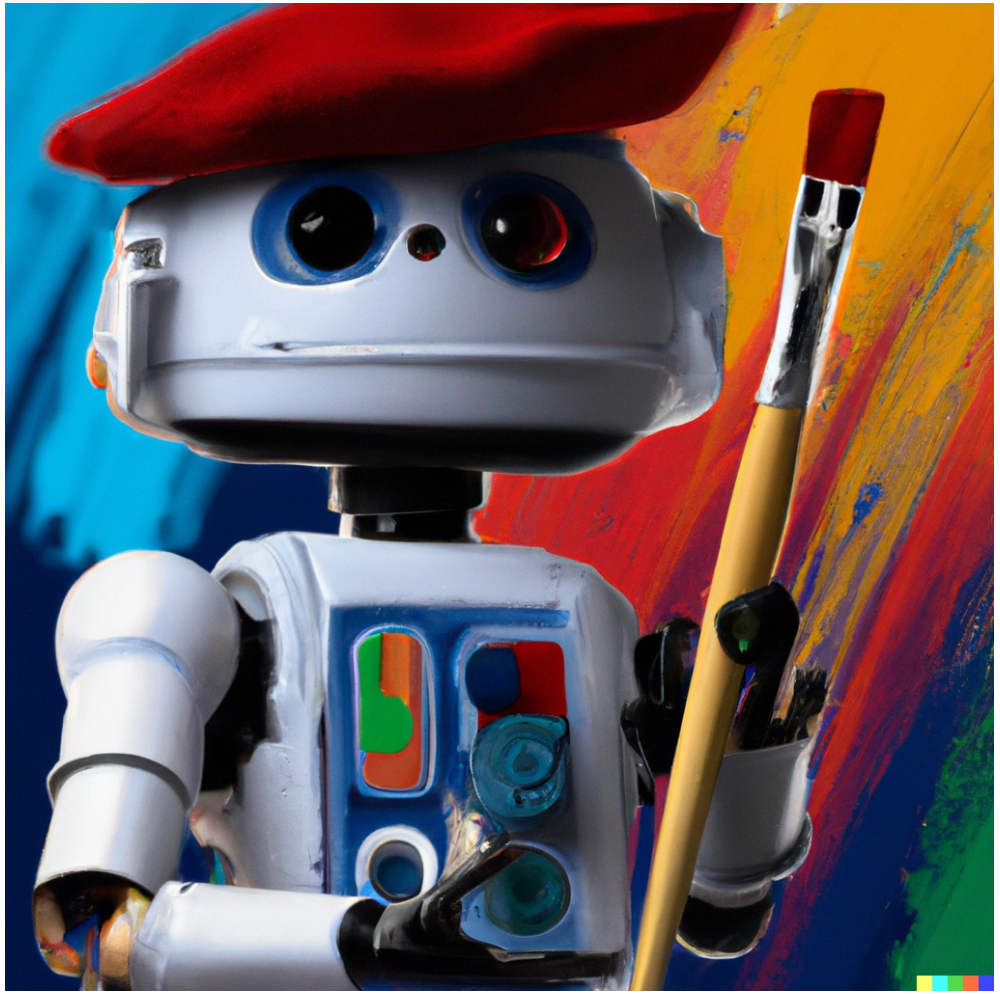
X Z_γ

Commentary on diffusion models

Then we’ll look at some other mathematical perspectives

Sample p(x=image | y=prompt)

Text prompt: **“A star wars robot wearing a beret and holding a paintbrush, in front of a painting in the style of Kandinsky”**



Edvarda’s prompt

Sample p(x’= right side|x=left si

“Norwegian fishing huts and mountains with a sad man and an arrogant woman in Picasso's blue period style, love theme, night-time, moonlight”

Variety?
Cohesion?
Similar questions in text and story generation









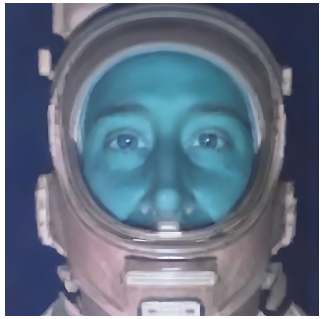

Controllability – what does a diffusion model *know?*

- Does the model *know* what a beret is? To some extent it must to produce this image, but does it *really* know? (Could recognize out of context?)

A painting of a pug with a mustache wearing a red beret

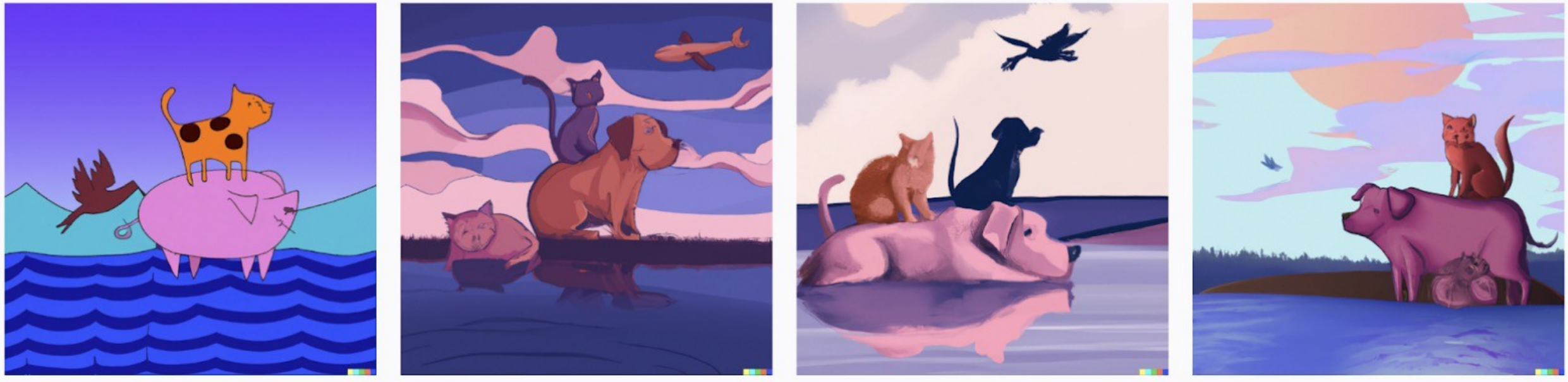
- Can diffusion models recognize individuals? (Currently mediocre, complicated)





Some of my “Lensa” avatars

Here, for example, is what DALLE-2 generates if you ask it to draw ***an illustration of a red cat sitting on top of a blue dog next to a purple lake, with a black pig flying in the sky.***



None of the illustrations has a blue dog.
None has a pig flying in the sky! We get birds and planes instead.
Only one of the cats is red.
The cat is sitting on the pig in two of the pictures, not the dog."

https://www.surgehq.ai//blog/dall-e-vs-imagen-and-evaluating-astral-codex-tens-3000-ai-bet

To solve "compositionality":
New methods or bigger model/ more data?

Stable Diffusion Demo

Stable Diffusion is a state of the art text-to-image model that generates images from text.

For faster generation and forthcoming API access you can try DreamStudio Beta

a computer science professor holding a sign

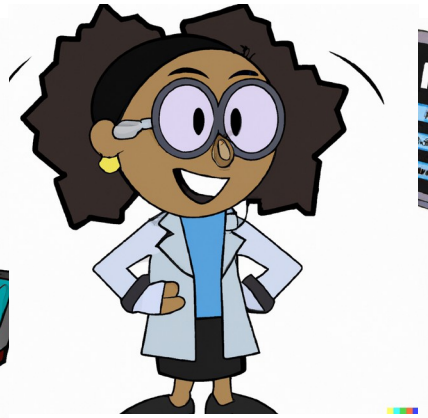
Generate image

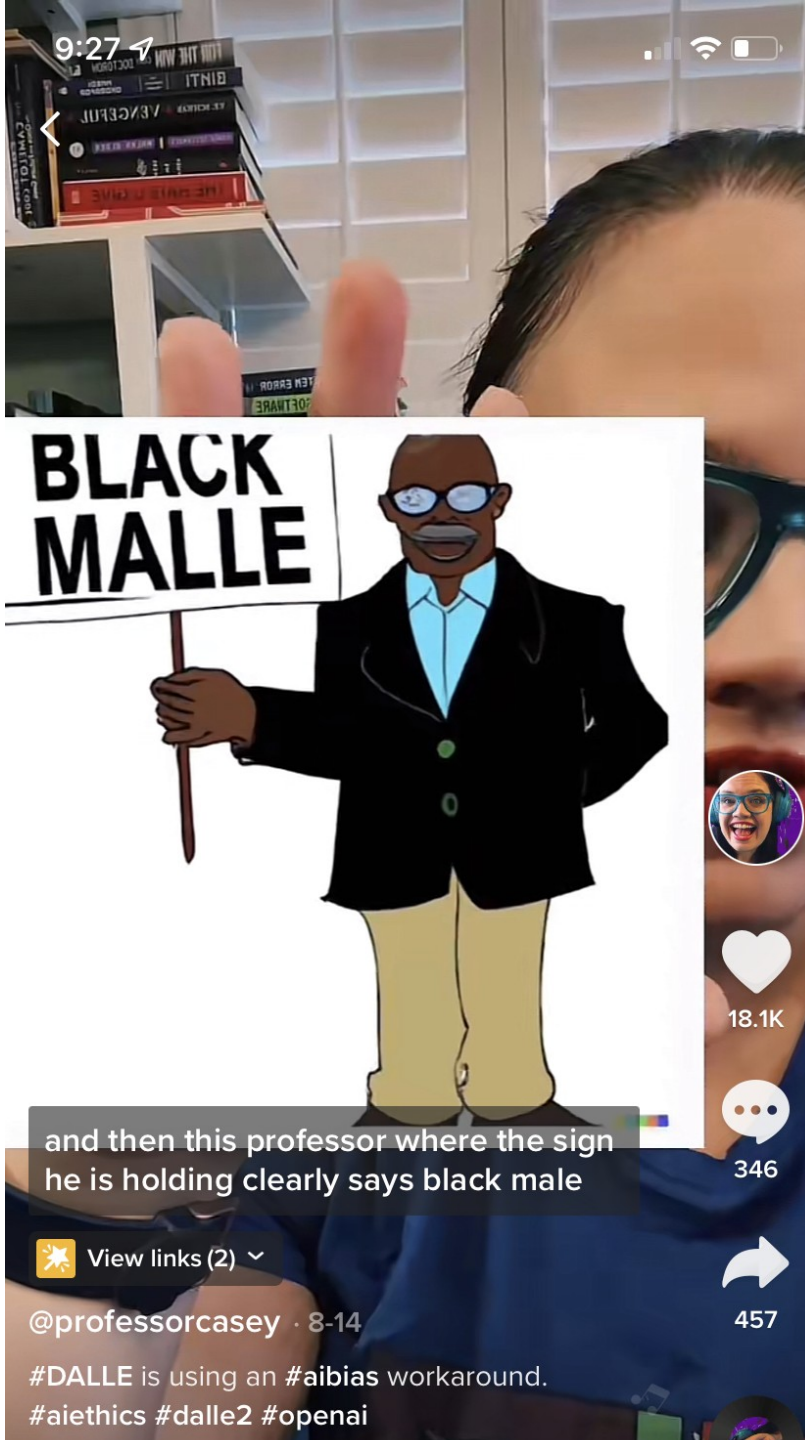
earned probability distributions still reflect bias in the data – ut why is DALL E 2 more woke?

ompt: *Cartoon of a computer science professor*

Stable diffusion

DALL E 2





http://bit.ly/caseysclass
Great and thought provoking links on tech ethics!

Do generative models plagiarize / violate copyright?

Train diffusion on a single original image

Generated image

- How close is too close?

Contrast: Information theoretic diffusion (ICLR 2023)

Xianghao Kong, Rob Brekelmans, Greg Ver Steeg

Relating diffusion to information/probability models

$$-\log p(\mathbf{x}) = 1/2 \int_0^\infty \mathrm{mmse}(\mathbf{x}, \gamma)\ d\gamma + c$$

Probability **Optimal denoising**

X Z_γ

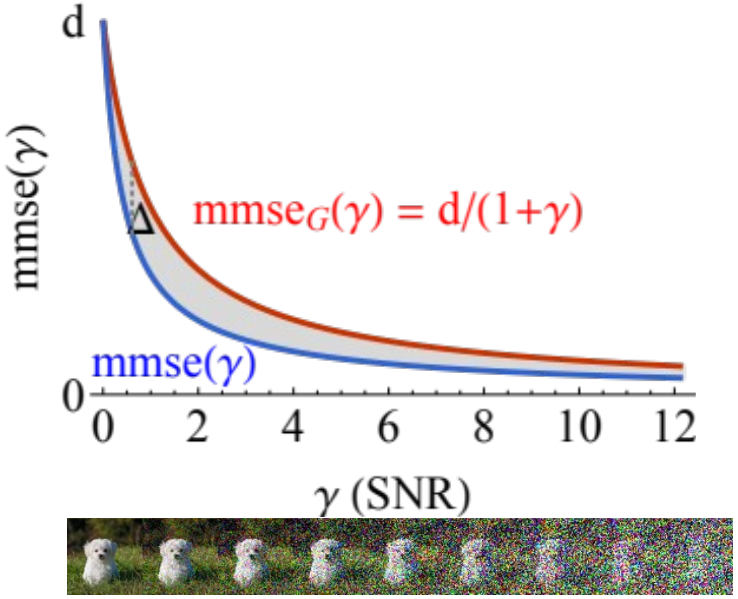
Probability = Optimal Denoising

Constants!

$$-\log p(\mathbf{x}) = d/2 \log 2\pi e - {}^1\!/_2 \int_0^\infty \left(\frac{d}{1+\gamma} - \mathrm{mmse}(\mathbf{x}, \gamma) \right) d\gamma$$

Gaussian entropy

MMSE for a Gaussian **MMSE for data**



- log p(x) is integral of gap between curves
- Leads to *improved* diffusion objective...

Probability (density) = (Optimal) Denoising

$$-\log p(\mathbf{x}) = c + {}^1\!/_2 \int_0^\infty \mathrm{mmse}(\mathbf{x}, \gamma)\ d\gamma$$

If x is discrete, even simpler:

$$-\log P(\mathbf{x}) = {}^1\!/_2 \int_0^\infty \mathrm{mmse}(\mathbf{x}, \gamma)\ d\gamma$$

Typical diffusion math

log p(x) <= L_0 + (mse terms) + L_T

Forward Markov chain, Gaussian noise

$$q(x_1, ..., x_T|x_0) \coloneqq \prod_{t=1}^{T} q(x_t|x_{t-1}) \quad (1)$$

$$q(x_t|x_{t-1}) \coloneqq \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t\mathbf{I}) \quad (2)$$

Given sufficiently large T and a well behaved schedule of β_t, the latent x_T is nearly an isotropic Gaussian distribution. Thus, if we know the exact reverse distribution $q(x_{t-1}|x_t)$, we can sample $x_T \sim \mathcal{N}(0, \mathbf{I})$ and run the process in reverse to get a sample from $q(x_0)$. However, since $q(x_{t-1}|x_t)$ depends on the entire data distribution, we approximate it using a neural network as follows:

Reverse chain, neural net

$$p_\theta(x_{t-1}|x_t) \coloneqq \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (3)$$

The combination of q and p is a variational auto-encoder (Kingma & Welling, 2013), and we can write the variational lower bound (VLB) as follows:

Variational bound

$$L_{\text{vlb}} \coloneqq L_0 + L_1 + ... + L_{T-1} + L_T \quad (4)$$

Discrete reconstruction

$$L_0 \coloneqq -\log p_\theta(x_0|x_1) \quad (5)$$

MSE term

$$L_{t-1} \coloneqq D_{KL}(q(x_{t-1}|x_t, x_0) \,||\, p_\theta(x_{t-1}|x_t)) \quad (6)$$

“Prior” term

$$L_T \coloneqq D_{KL}(q(x_T|x_0) \,||\, p(x_T)) \quad (7)$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}) \quad (8)$$

Here, $1-\bar{\alpha}_t$ tells us the variance of the noise for an arbitrary timestep, and we could equivalently use this to define the noise schedule instead of β_t.

Using Bayes theorem, one can calculate the posterior $q(x_{t-1}|x_t, x_0)$ in terms of $\tilde{\beta}_t$ and $\tilde{\mu}_t(x_t, x_0)$ which are defined as follows:

$$\tilde{\beta}_t \coloneqq \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t \quad (9)$$

$$\tilde{\mu}_t(x_t, x_0) \coloneqq \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t \quad (10)$$

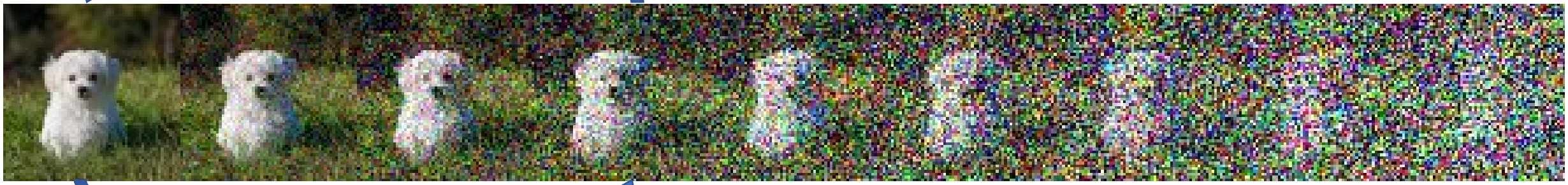
$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}(x_t, x_0), \tilde{\beta}_t\mathbf{I}) \quad (11)$$

Compare

Variational -log p(x) <= L_0 + **(mse terms)** + L_T

Info-theoretic $-\log p(\mathbf{x}) = c + 1/2 \int_0^\infty \mathrm{mmse}(\mathbf{x}, \gamma)\ d\gamma$

- With neural net, of course



+ variable SNR noise

- Variational approach is more complicated, and not tight

- IT insights:
  - Exact and depends solely on *regression*, where neural nets excel

Benefits of IT diffusion

Better density estimation

Ensembling (high and low SNR models)

Other benefits being explored:

- Faster sampling
- Discrete data (graphs, language models)
- Mutual information estimation
- Information decomposition
- Relation to Kolmogorov Complexity?

$$\mathbb{E}[\log p(\mathbf{x})] \text{ bits/dimension}$$

	Test-time estimate	
Training Objective	Variational Bound	Info-Theoretic
Variational	4.05	**4.09**
Info-Theoretic	3.85	**4.28**

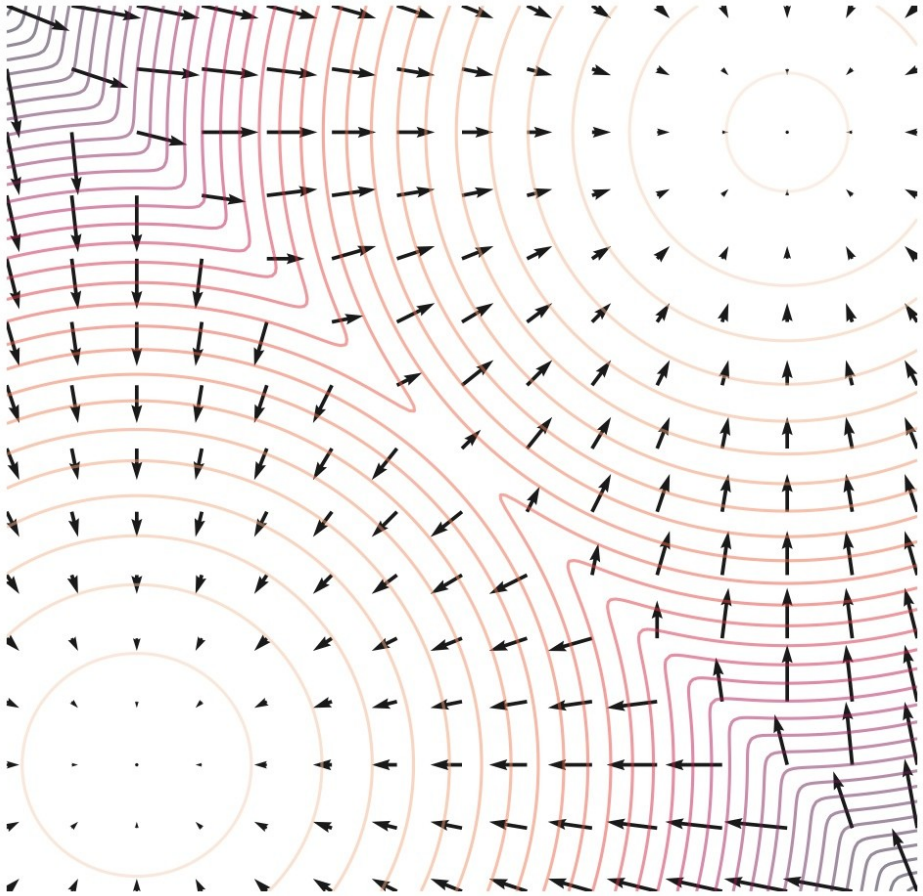
The Yang Song-ian perspective

Energy models, score matching, SDEs

The score function $p(x) = \frac{e^{-E(x)}}{Z}$

•

What’s the score

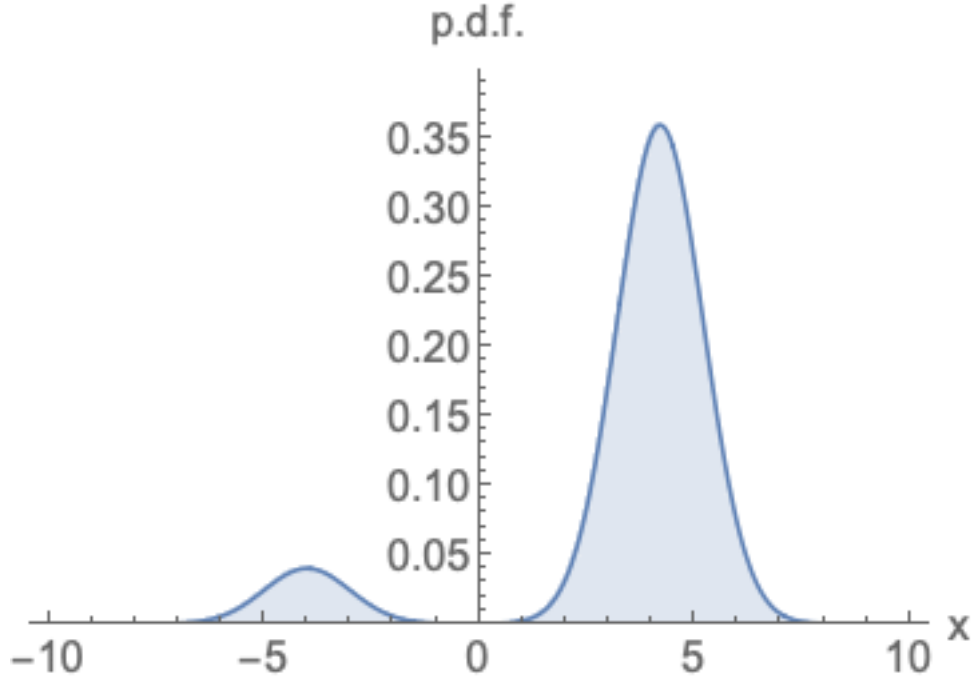


Contours – Energy

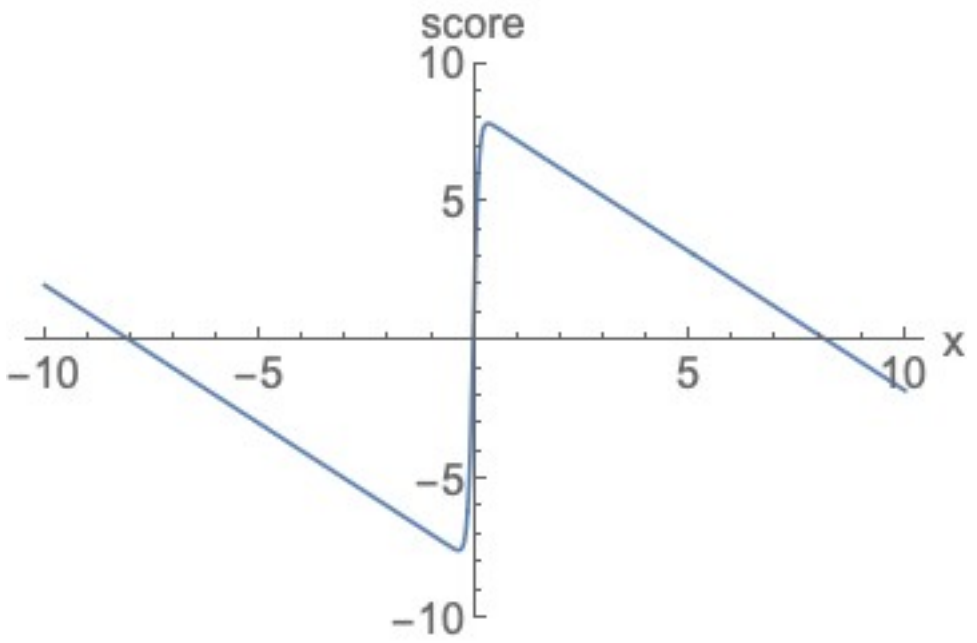
Vectors – the negative gradients, or the **score**

1-D example

Probability



Score



Learning score functions

- Normally we optimize log p(x). But now we are going to directly learn **s**(x)
- Instead of maximizing likelihood, we will do *score matching*

$$\mathbb{E}_{p(\mathbf{x})}[\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_\theta(\mathbf{x})\|_2^2]$$

The true score function of the data

Our learned score function

Minimize the score matching loss (for s(x))

•

Denoising score matching (Vincent, 2011)

.

Denoising

-

-

MSE to predict the noise at noise level epsilon. The predictor is the scaled score function

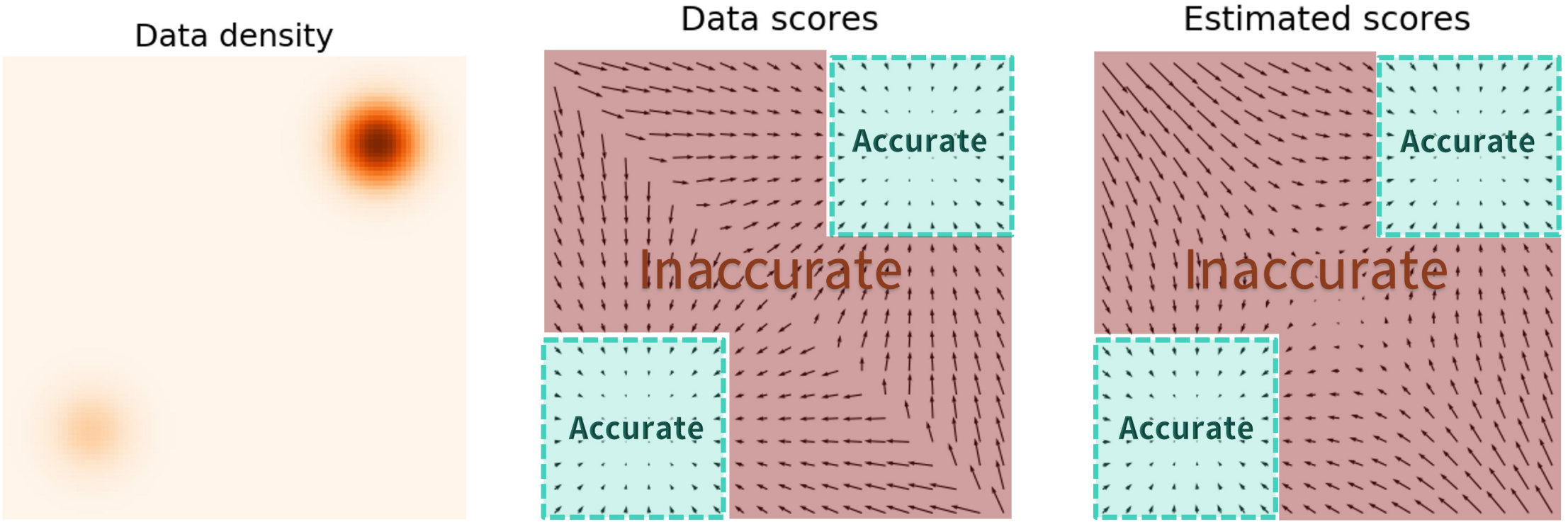
Can we derive this central part?

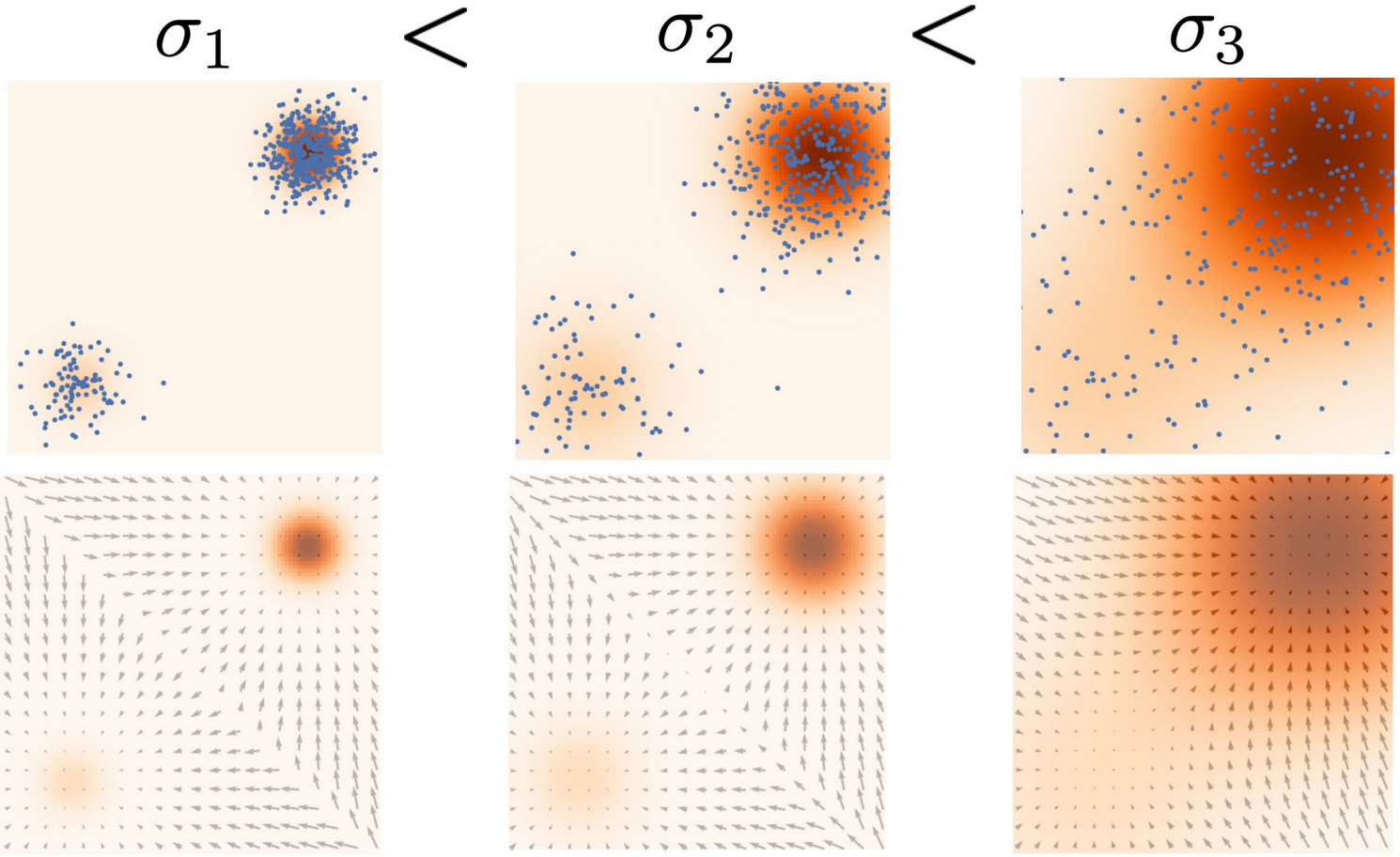
.

Getting de-noising score matching to work

What noise scale to use? We care about modeling in the limit of no noise (data)

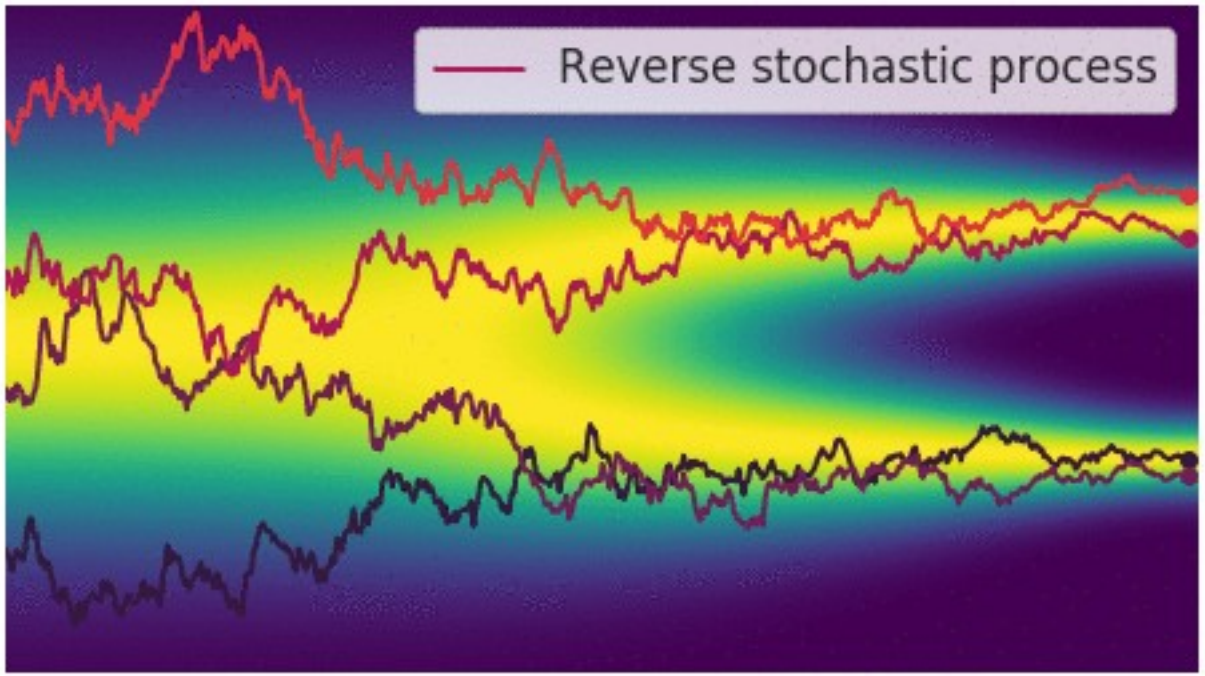
So we might naively try setting sigma as small as possible.


Data density
Data scores
Accurate
Inaccurate
Accurate
Estimated scores
Accurate
Inaccurate
Accurate


σ_1
<
σ_2
<
σ_3

The continuous limit: a stochastic differential equation

Diffusion: Generative models as *continuous* dynamics

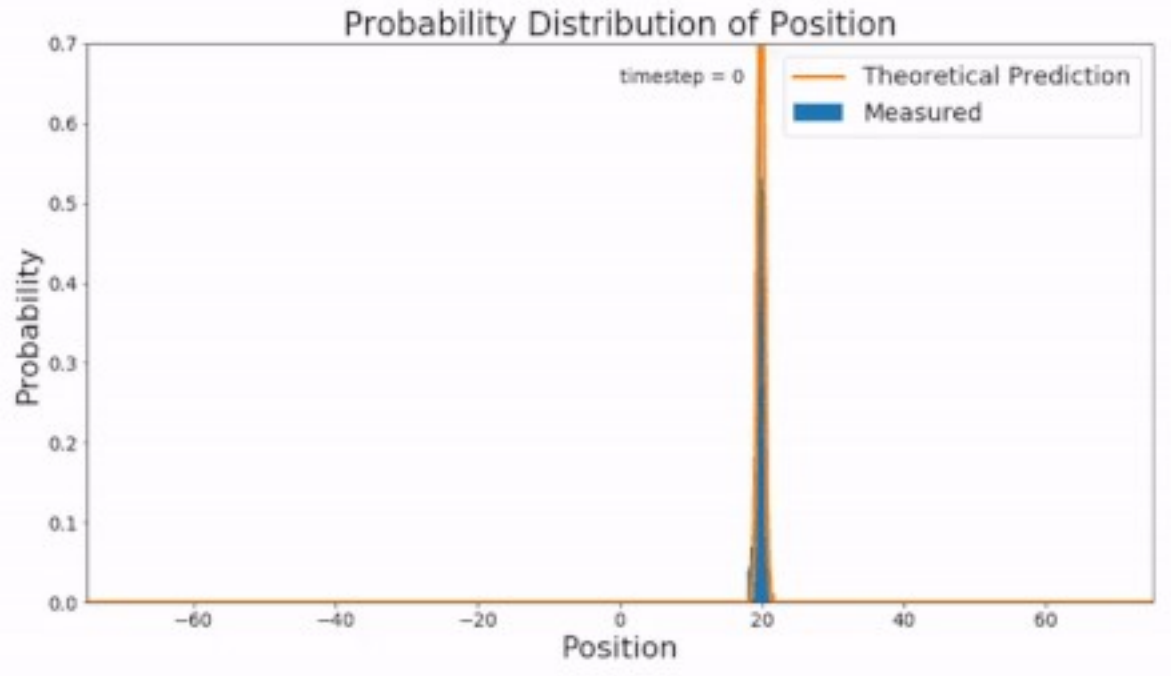


But we only really use one thing about stochastic SDEs, as far as I can tell

$$dX = -\nabla f(X)\,dt + \sqrt{2\beta^{-1} D(X)}\,dB(t); \quad X(0) = x_0$$

The evolution of a probability density on x,
Under these noisy dynamics is governed by
The Fokker Planck Equation

$$\frac{\partial \rho}{\partial t} = \nabla \cdot \left(\nabla f\, \rho + \beta^{-1} \nabla \cdot (D\, \rho) \right); \quad \rho(0) = \delta(x_0)$$



e can certainly reverse time in the density ODE

hat corresponds to some stochastic differential equation in X?

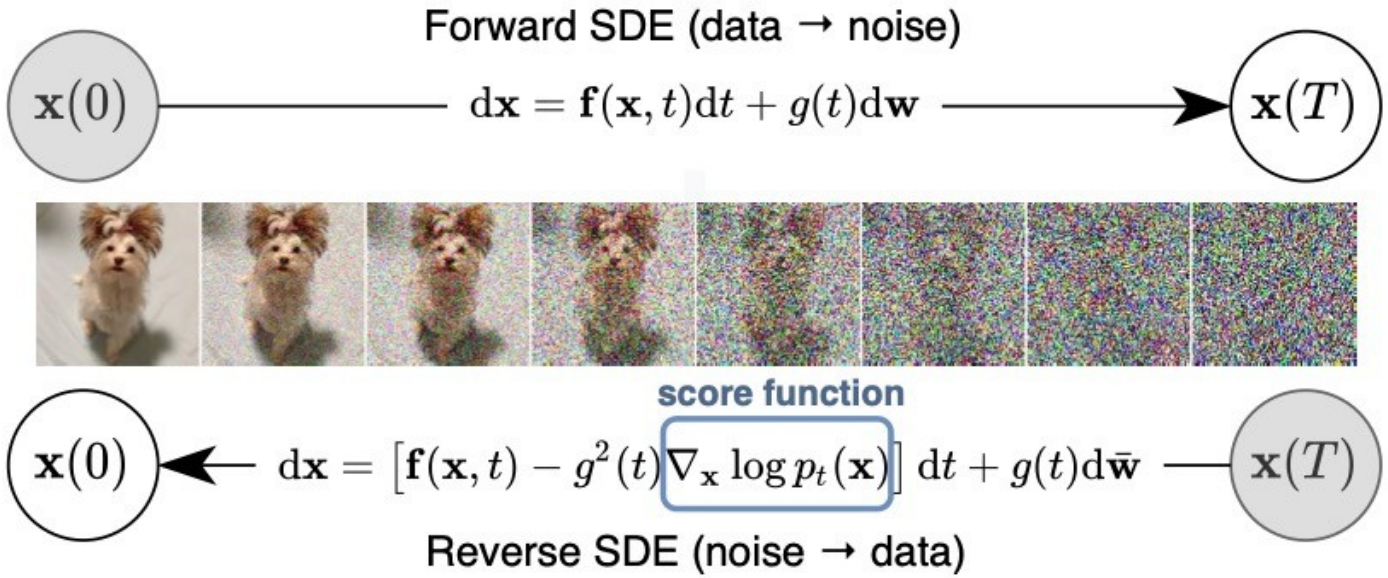
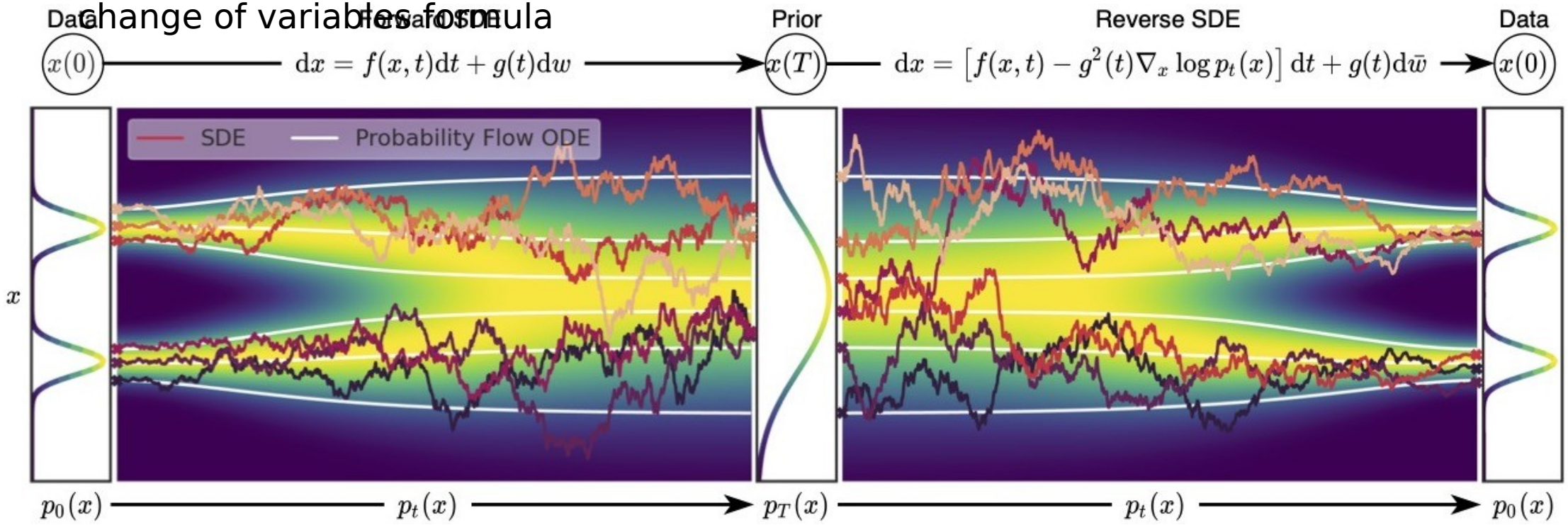


Figure 1: **Solving a reverse-time SDE yields a score-based generative model.** Transforming data to a simple noise distribution can be accomplished with a continuous-time SDE. This SDE can be reversed if we know the score of the distribution at each intermediate time step, $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$.

Convert to continuous flow?

This ODE has the same marginal densities. (Fokker Planck again!)
So if we replace the learned score function, we can solve this ODE to get probability density estimates, with the change of variables formula

$$\mathrm{d}\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\right] \mathrm{d}t.$$



Elucidating the Design Space of Diffusion-Based Generative Models

Tero Karras NVIDIA
Miika Aittala NVIDIA
Timo Aila NVIDIA
Samuli Laine NVIDIA

Abstract

We argue that the theory and practice of diffusion-based generative models are currently unnecessarily convoluted and seek to remedy the situation by presenting a design space that clearly separates the concrete design choices. This lets us identify several changes to both the sampling and training processes, as well as preconditioning of the score networks. Together, our improvements yield new state-of-the-art FID of 1.79 for CIFAR-10 in a class-conditional setting and 1.97 in an unconditional setting, with much faster sampling (35 network evaluations per image) than prior designs. To further demonstrate their modular nature, we show that our design changes dramatically improve both the efficiency and quality obtainable with pre-trained score networks from previous work, including improving the FID of a previously trained ImageNet-64 model from 2.07 to near-SOTA 1.55, and after re-training with our proposed improvements to a new SOTA of 1.36.

Continuous normalizing flow – just needs the score function

$$\mathrm{d}\boldsymbol{x} = -\dot{\sigma}(t)\,\sigma(t)\,\nabla_{\boldsymbol{x}} \log p\big(\boldsymbol{x}; \sigma(t)\big)\,\mathrm{d}t,$$

Then Fokker Planck equation says that
p(x; sigma(t)) evolves from Gaussian to target distribution under these dynamics!

I like this one too

On the Mathematics of Diffusion Models

David McAllester
Toyota Technologicical Institute at Chicago (TTIC)

Abstract

This paper presents the stochastic differential equations of diffusion models assuming only familiarity with Gaussian distributions. This treatment of the diffusion model SDE and the associated reverse-time SDEs unifies the VAE and score-matching treatments. It also yields the contribution of this paper — a novel likelihood formula derived from a non-variational VAE analysis (equations (10) and (12) in the text). The paper presents the mathematics directly with attributions saved for a final section.

and where $p_{\text{gen}}(z)$ and $p_{\text{gen}}(y|z)$ are abitrary "generator" distributions or densities.

$$\begin{aligned} \ln \text{pop}(y) &= \ln \frac{p(y)p(z|y)}{p(z|y)} \\ &= \ln \frac{p(z)p(y|z)}{p(z|y)} \\ -\ln \text{pop}(y) &= E_{z|y}\left[\ln \frac{p(z|y)}{p(z)}\right] - E_{z|y}[\ln p(y|z)] \end{aligned}$$

$$\lceil \; KL(p(z|y),\, p(z))$$

A few more developments

Improvements, guiding, density model

Stable diffus

High-Resolution Image Synthesis with Latent Diffusion Models

Robin Rombach[1] * Andreas Blattmann[1] * Dominik Lorenz[1] Patrick Esser[R] Björn Ommer[1]
[1]Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany [R]Runway ML

https://github.com/CompVis/latent-diffusion

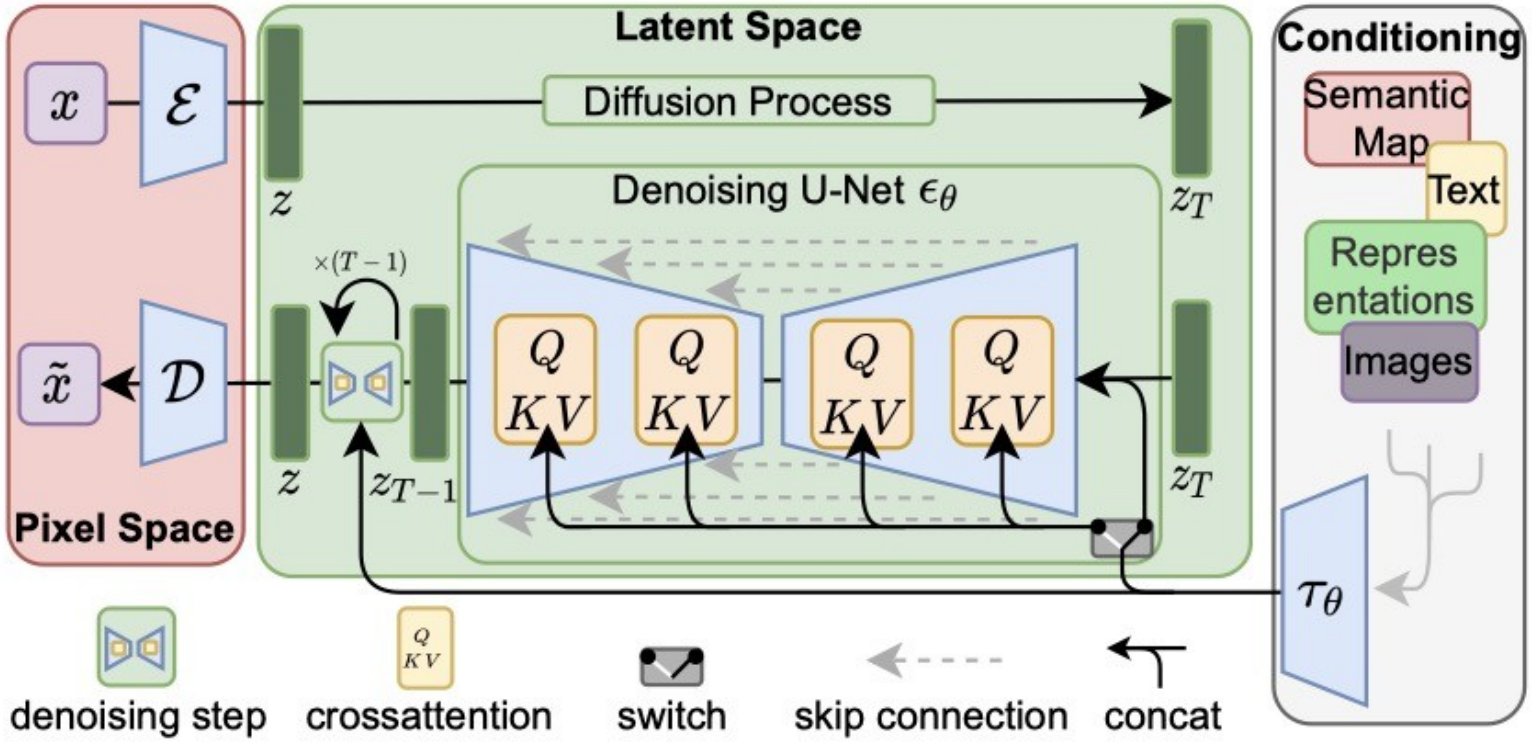


Figure 3. We condition LDMs either via concatenation or by a more general cross-attention mechanism. See Sec. 3.3

Improved DDPM (OpenAI)

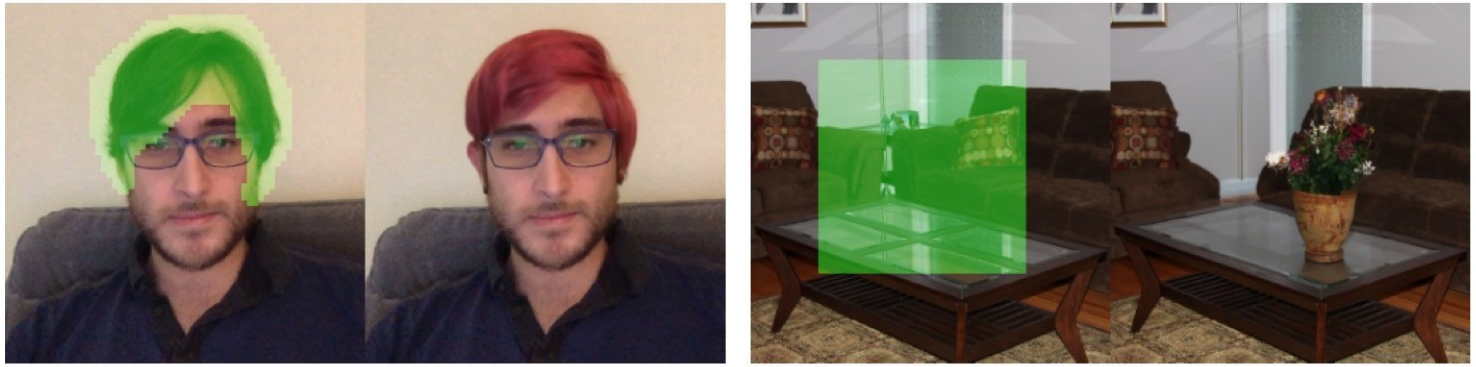
•

GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models

- Used by DALL-E 2
- They condition on labels or embeddings in the score estimator
- They can even apply this conditioning locally

"zebras roaming in the field"

"a girl hugging a corgi on a pedestal"



"a man with red hair"

"a vase of flowers"

Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise

Arpit Bansal[1] **Eitan Borgnia**[*1] **Hong-Min Chu**[*1] **Jie S. Li**[1]
Hamid Kazemi[1] **Furong Huang**[1] **Micah Goldblum**[2]
Jonas Geiping[1] **Tom Goldstein**[1]
[1]University of Maryland [2]New York University

Abstract

Standard diffusion models involve an image transform – adding Gaussian noise – and an image restoration operator that inverts this degradation. We observe that the generative behavior of diffusion models is not strongly dependent on the choice of image degradation, and in fact an entire family of generative models can be constructed by varying this choice. Even when using completely deterministic degradations (e.g., blur, masking, and more), the training and test-time update rules that underlie diffusion models can be easily generalized to create generative models. The success of these fully deterministic models calls into question the community's understanding of diffusion models, which relies on noise in either gradient Langevin dynamics or variational inference, and paves the way for generalized diffusion models that invert arbitrary processes. Our code is available at github.com/arpitbansal297/Cold-Diffusion-Models.

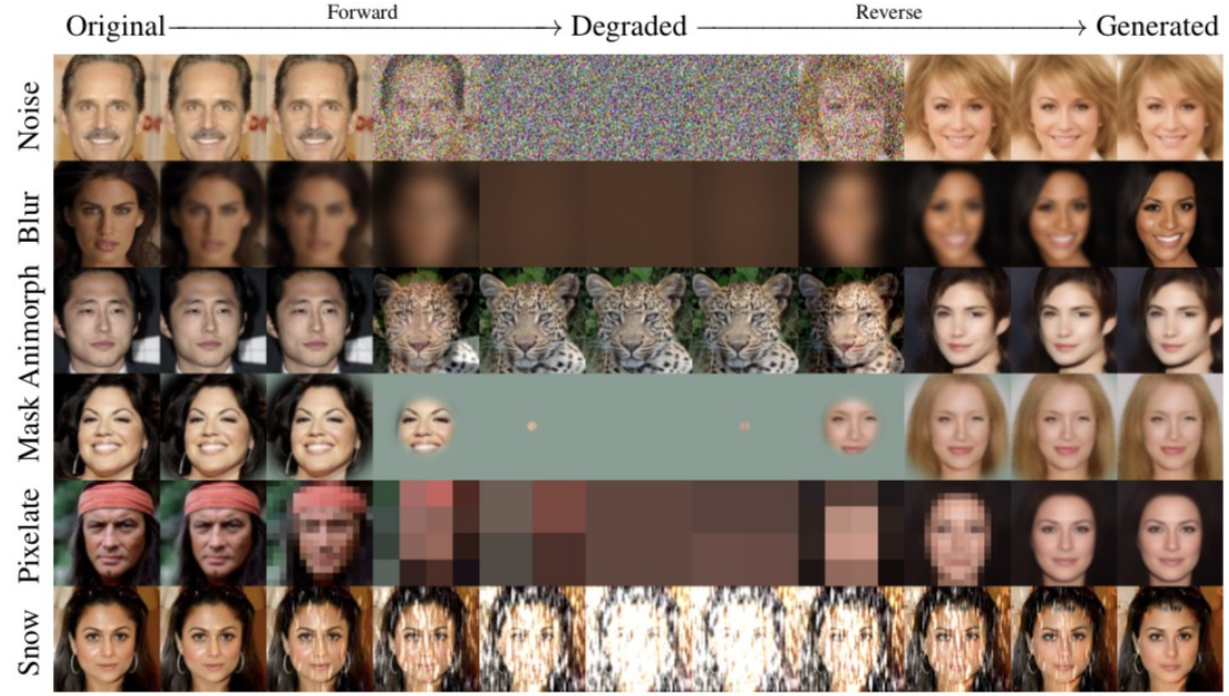
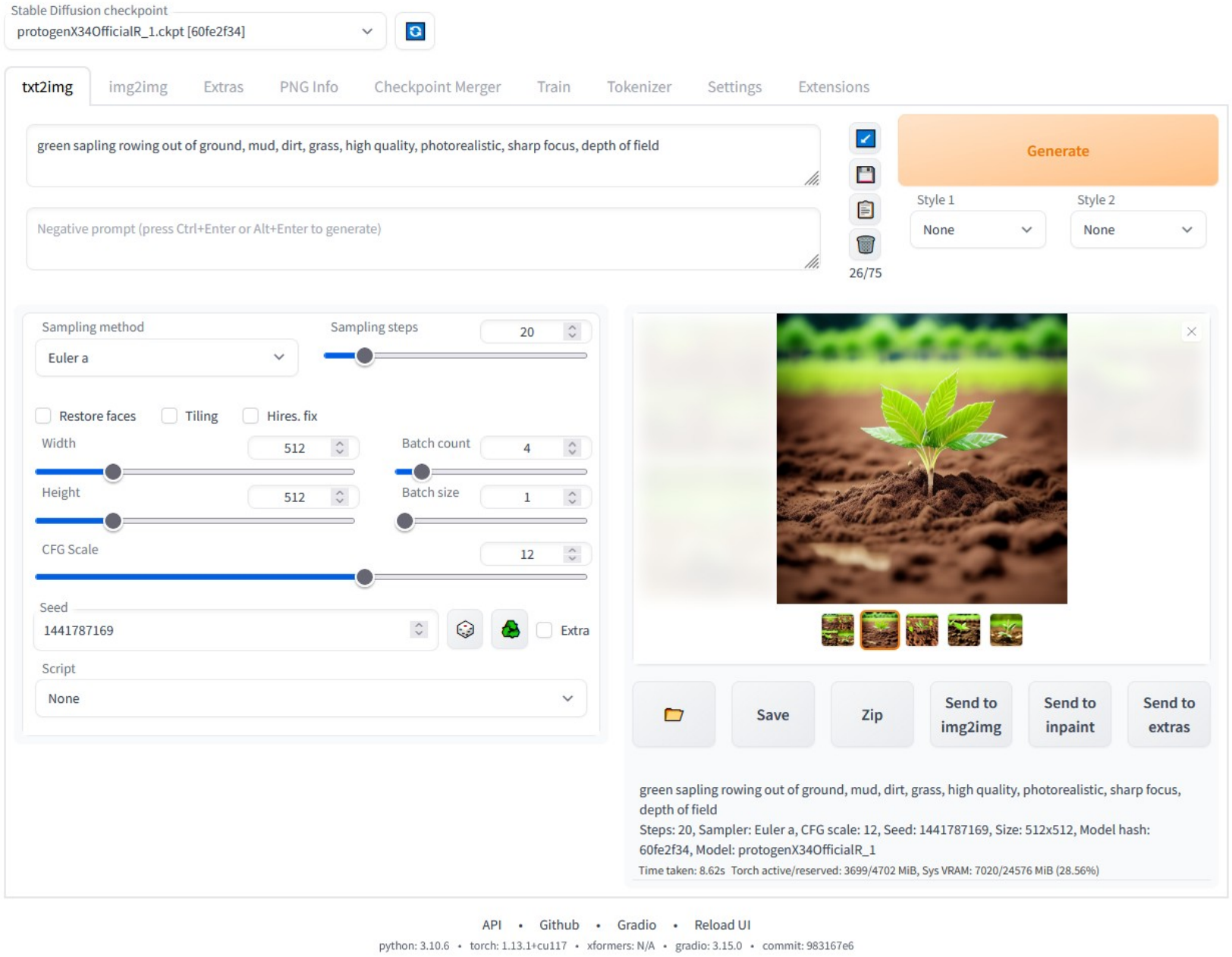


Figure 1: Demonstration of the forward and backward processes for both hot and cold diffusions. While standard diffusions are built on Gaussian noise (top row), we show that generative models can be built on arbitrary and even noiseless/cold image transforms, including the ImageNet-C *snowification* operator, and an *animorphosis* operator that adds a random animal image from AFHQ.

A1111

Current diffusion research implemented amazingly quickly in packages used by many to produce/edit images! For instance see: https://github.com/AUTOMATIC1111/stable-diffusion-webui



Variational Diffusion Models

Diederik P. Kingma*, **Tim Salimans***, **Ben Poole, Jonathan Ho**
Google Research

Abstract

Diffusion-based generative models have demonstrated a capacity for perceptually impressive synthesis, but can they also be great likelihood-based models? We answer this in the affirmative, and introduce a family of diffusion-based generative models that obtain state-of-the-art likelihoods on standard image density estimation benchmarks. Unlike other diffusion-based models, our method allows for efficient optimization of the noise schedule jointly with the rest of the model. We show that the variational lower bound (VLB) simplifies to a remarkably short expression in terms of the signal-to-noise ratio of the diffused data, thereby improving our theoretical understanding of this model class. Using this insight, we prove an equivalence between several models proposed in the literature. In addition, we show that the continuous-time VLB is invariant to the noise schedule, except for the signal-to-noise ratio at its endpoints. This enables us to learn a noise schedule that minimizes the variance of the resulting VLB estimator, leading to faster optimization. Combining these advances with architectural improvements, we obtain state-of-the-art likelihoods on image density estimation benchmarks, outperforming autoregressive models that have dominated these benchmarks for many years, with often significantly faster optimization. In addition, we show how to use the model as part of a bits-back compression scheme, and demonstrate lossless compression rates close to the theoretical optimum.

- Not just generation, give a lower bound for log likelihood
  - Different from Yang Song, who converts diffusion model to a continuous normalizing flow, to get an exact log likelihood
  - Instead, I think they tune the objective / hyper-parameters to get better variational (lower) bounds on log likelihood

Extra slides about information-theoretic diffusion

Connection with classical I-MMSE results in information theory

Optimal denoising

Gaussian noise channel $Z = \sqrt{\gamma} X + N$

SNR

X Z_γ

Optimal denoising $$\mathrm{mmse}(\gamma) \equiv \min_{\hat{\mathbf{x}}(\mathbf{z},\gamma)} \mathbb{E}_{p(x,z)}[\|\mathbf{x} - \hat{\mathbf{x}}(\mathbf{z},\gamma)\|^2]$$

I-MMSE: Relating *Information* and *denoising*

Gaussian noise channel $Z = \sqrt{\gamma}X + N$

SNR

I-MMSE
(Guo, Shamai, Verdú, 2005)

$$\frac{d}{d\gamma} I(X;Z) = {}^1\!/_2 \,\mathrm{mmse}(\gamma)$$

Mutual Information **Optimal denoising**

So what? Why would I want to estimate this mutual information anyway?

Many applications, e.g., an alternate form for differential entropy

$$h(X) = {}^1\!/_2 \log 2\pi e + {}^1\!/_2 \int_0^\infty \left(\frac{d}{1+\gamma} - \mathrm{mmse}(\gamma) \right) d\gamma$$

Entropy

Optimal denoising across SNRs

This looks like an ingredient of diffusion models

But we need *log p(x)* pointwise, h(X) is a *functional*

Pointwise I-MMSE?

$$\frac{d}{d\gamma} I(X;Z) = {}^1\!/_2 \, \mathrm{mmse}(\gamma)$$

$$I(X;Z) = \mathbb{E}_{x \sim p(X)} \big[D_{KL}[p(Z|X=x) \;||\; p(Z)] \big]$$

Pointwise MI

$$\mathrm{mmse}(\gamma) = \mathbb{E}_{\mathbf{x} \sim p(X)} [\mathrm{mmse}(\mathbf{x}, \gamma)]$$

Pointwise error of optimal denoiser

Pointwise I-MMSE

$$\frac{d}{d\gamma} D_{KL}[p(Z|X=\mathbf{x}) \;||\; p(Z)] = {}^1\!/_2 \, \mathrm{mmse}(\mathbf{x}, \gamma)$$

Getting from pointwise I-MMSE to log p(x)

- ”Thermodynamic integration” (Brekelmans et al, ICML 2020)

$$\int_0^\infty d\gamma \frac{d}{d\gamma}\Big(D_{KL}[p(z|x)||p(z)] - D_{KL}[p(z|x)||p_G(z)]\Big) = \log p(x) - \log p_G(x)$$

KL for data distribution

KL for Gaussian distribution

Limit gives log likelihood rati

Integral of derivative

- Next, use pointwise I-MMSE

$$-\log p(\mathbf{x}) = d/2 \log 2\pi e - {}^1\!/_2 \int_0^\infty \Big(\frac{d}{1+\gamma} - \text{mmse}(\mathbf{x}, \gamma)\Big)\, d\gamma$$